Unidad 1:

Introducción

La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento y posterior operación.
La ingeniería de software surge porque desarrollar software es una actividad mucho más compleja que simplemente escribir código.

Un sistema de software incluye el código fuente, la documentación, la arquitectura, los modelos de datos, los requerimientos y todos los artefactos necesarios para construir, mantener y evolucionar un producto.

A diferencia de la manufactura tradicional, el software representa conocimiento organizado en distintos niveles de abstracción y cada producto suele resolver un problema diferente, por lo que rara vez puede repetirse exactamente el mismo proceso.

Problemas del desarrollo de software

Históricamente, muchos proyectos fracasan por problemas relacionados con la especificación y la gestión más que por dificultades técnicas de programación. Los problemas más frecuentes son:

- ➢ El producto final no satisface las necesidades del cliente.
- ➢ Es difícil extenderlo o adaptarlo.
- ➢ Agregar nuevas funcionalidades resulta costoso.
- ➢ La documentación es insuficiente.
- ➢ La calidad es baja.
- ➢ Los tiempos y costos superan lo presupuestado.

Software

En términos generales, el software es un conjunto de programas junto con toda la documentación que los acompaña. Existen tres grandes categorías:

a. Software de sistema: encargado de administrar el hardware y brindar servicios básicos.
b. Utilitarios: realizan tareas específicas de soporte.
c. Software de aplicación: orientado a resolver necesidades del usuario.

El software como niveles de abstracción
El software puede entenderse como conocimiento representado en distintos niveles de abstracción. Cada nivel se apoya sobre el anterior.



1. La idea: Es la necesidad, problema del mundo real o visión del usuario expresada de forma intangible. En este nivel, el conocimiento se representa mediante requerimientos del usuario y modelos del dominio del problema, enfocado exclusivamente en qué debe hacer el sistema.

2. Cómo se hace: Es la estructuración lógica de la solución. Traduce la idea en modelos abstractos, arquitectura de software, patrones de diseño, componentes e interfaces (como diagramas UML). Permite razonar sobre la organización global y el comportamiento del sistema antes de la implementación.

3. El código fuente: Es la expresión concreta del diseño traducida a un lenguaje de programación legible para los desarrolladores (como Java, Python o C#). En esta etapa, las abstracciones arquitectónicas toman la forma de algoritmos, clases, métodos y estructuras de datos.

4. Las herramientas de desarrollo: Comprende el conjunto de compiladores, intérpretes, entornos de desarrollo (IDE), middleware y bibliotecas. Su función es procesar el código fuente, optimizarlo y proporcionar el soporte necesario para que interactúe con la plataforma de ejecución.

5. El lenguaje de máquina: Es la representación en código objeto o formato binario (instrucciones en ceros y unos). Representa la secuencia exacta de operaciones lógicas y aritméticas que la arquitectura del procesador comprende de forma nativa.

6. El hardware físico: La infraestructura física compuesta por el procesador (CPU), la memoria RAM, los circuitos integrados, los buses y los periféricos. Sin software, el hardware es una máquina inerte; al recibir las instrucciones en lenguaje de máquina, los transistores y circuitos ejecutan los cambios de estado eléctrico que dan vida a la idea inicial.

La mayor dificultad suele encontrarse en los niveles superiores, donde todavía debe entenderse qué necesita el usuario antes de traducir esa idea a una implementación concreta.

La construcción de un sistema atraviesa una serie de transformaciones que convierten una idea en un producto funcionando.

1. **Requerimientos**: Documentación de las necesidades.
2. **Prototipos**: Representaciones tempranas e interactivas.
3. **Diseño**: Arquitectura y modelo de datos.
4. **Implementación**: Desarrollo del código fuente.
5. **Despliegue**: Puesta en funcionamiento.
6. **Ambiente de producción**: Operación del sistema real.

Cada etapa reduce la incertidumbre y permite validar decisiones antes de avanzar hacia la siguiente.

Software y manufactura

Aunque ambos producen un resultado final, el desarrollo de software no puede analizarse como una línea de producción tradicional. Las principales diferencias son:

- ➢ El software es menos predecible.
- ➢ No hay producción en masa, casi ningún producto de software es igual a otro.
- ➢ No todas las fallas corresponden a errores de programación.
- ➢ El software no se gasta con el uso.
- ➢ No está limitado directamente por las leyes de la física, sino por restricciones conceptuales.
- ➔ *Análisis del ensayo "No hay balas de plata"*

La idea principal del trabajo es que no existe una tecnología capaz de multiplicar por diez la productividad del desarrollo resolviendo el problema fundamental del software. Los grandes avances históricos eliminaron dificultades secundarias, pero no la verdadera complejidad del diseño.

No existe ninguna "bala de plata" tecnológica que resuelva la complejidad esencial. La etapa más difícil del software es la especificación conceptual y el diseño, no el trabajo manual de programar.

El modelo aristotélico

La parte difícil del software no es escribir código, sino comprender qué debe construirse y diseñarlo correctamente. Brooks distingue dos componentes del desarrollo de software.

1. **El accidente**

Corresponde a las dificultades tecnológicas de representación, es decir, problemas que acompañan la construcción pero no forman parte del problema esencial. Ejemplos: sintaxis compleja, limitaciones de memoria, tiempos de compilación, herramientas poco eficientes. Estas dificultades pueden reducirse mediante mejores lenguajes o herramientas.

2. **La esencia**

La esencia corresponde a la construcción de estructuras conceptuales complejas: reglas de negocio, algoritmos, relaciones entre componentes y decisiones de diseño.

Es la dificultad inherente de entender y modelar lo que el usuario realmente quiere. Sus características principales son la mutabilidad constante del software, su naturaleza invisible y la necesidad de acoplarse a sistemas humanos ilógicos.

Escribir código no es el problema real. La parte difícil es la especificación, el diseño y la validación de la estructura conceptual.

Las cuatro propiedades irreducibles

Brooks sostiene que la esencia del software posee cuatro características que resisten la automatización.

- ➢ Complejidad: A medida que un sistema crece, las interacciones entre sus componentes aumentan de forma no lineal.
  "El software está formado por múltiples elementos que interactúan entre sí. Al aumentar su tamaño, no solo aumentan los elementos, sino también sus relaciones y posibles comportamientos, por lo que la complejidad crece de manera no lineal. Además, cada parte puede presentar problemas particulares que no se resuelven simplemente repitiendo soluciones anteriores." *Chat gpt*
- ➢ Conformidad: El software debe adaptarse a restricciones externas impuestas por usuarios, organizaciones, leyes o sistemas existentes.
- ➢ Mutabilidad: Los requerimientos cambian constantemente, por lo que el software permanece en evolución durante toda su vida útil.
- ➢ Invisibilidad: El software carece de una representación física natural, lo que dificulta comprender su estructura completa de un vistazo.

La historia elimina lo accidental

Los grandes saltos tecnológicos aumentaron la productividad, pero todos atacaron únicamente el esfuerzo accidental. Ninguno de estos avances eliminó la complejidad esencial del diseño.

Época	Avance
Años 60	Lenguajes de alto nivel
Años 70	Time-sharing
Años 80 y 90	Entornos integrados (IDE)

IA y desarrollo de software

Las herramientas de inteligencia artificial aceleran enormemente la generación de código, pero siguen dependiendo de la capacidad humana para especificar correctamente el problema.

- El embudo de la especificación

El proceso puede entenderse como un embudo.

1. La intención del negocio.
2. La traducción de esa intención en especificaciones claras.
3. La representación final mediante código.

El verdadero cuello de botella continúa siendo la capacidad humana para definir correctamente qué debe construirse.



- Cómo atacar la esencia

Dado que la esencia no puede eliminarse, Brooks propone reducir su impacto mediante buenas prácticas.

- **Buy vs. Build — Comprar frente a construir**: Comprar soluciones cuando resulte conveniente. La integración es más barata que generar código nuevo.
- **Rapid Prototyping — Prototipado rápido**: Descubrir requerimientos mediante iteraciones. La IA solo sirve acá como herramienta de descarte rápido para refinar la especificación.
- **Grow, not build — Hacer crecer, no construir todo de una vez**: Hacer crecer un sistema funcional paso a paso. Ir engrosando el sistema orgánicamente, manteniendo siempre un sistema funcional.
- **Grandes diseñadores**: La calidad del diseño sigue dependiendo del talento humano.

La IA puede asistir en tareas accidentales, pero el diseño continúa siendo una actividad fundamentalmente humana.

Factores de éxito y fracaso

Diversos estudios muestran que el éxito de un proyecto depende más de la comunicación y la gestión que de la programación.

- Cuando un proyecto tiene éxito
  - ★ Involucramiento del usuario: 15,9 %
  - ★ Apoyo de la gerencia: 13,0 %
  - ★ Requerimientos claros: 9,6 %
  - ★ Planeamiento adecuado: 8,2 %
  - ★ Expectativas realistas: 7,7 %
  - ★ Hitos intermedios: 7,7 %
  - ★ Personas competentes: 7,2 %

- Principales causas de fracaso
  - ★ Requerimientos incompletos 13,1 %
  - ★ Poco involucramiento del usuario 12,4 %
  - ★ Falta de recursos 10,6 %
  - ★ Expectativas poco realistas 9,3 %
  - ★ Poco apoyo gerencial 8,7 %

★ Cambios constantes de requerimientos 8,1 %

Ingeniería de Software

Existen varias definiciones aceptadas.

- ➢ Parnas: Multi-person construction of multi-version software.
- ➢ IEEE: Aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software.
- ➢ Sommerville: Disciplina que abarca todos los aspectos de la producción de software, desde la concepción inicial hasta su operación y mantenimiento.

El elemento común de todas estas definiciones es que la ingeniería de software estudia el desarrollo organizado de sistemas que evolucionan a lo largo del tiempo.

SWEBOK

El Software Engineering Body of Knowledge (SWEBOK) es el cuerpo de conocimientos elaborado por IEEE que organiza la disciplina en áreas de conocimiento.

Dentro del contexto de la materia aparecen varios bloques relacionados con la ingeniería de sistemas.

a. **Fundamentos**: pensamiento sistémico, ciencia de sistemas, representación mediante modelos.
b. **Gestión del ciclo de vida**: definición del concepto, definición del sistema, realización, despliegue, operación, gestión del ciclo de vida, estándares.
c. **Aplicaciones**: sistemas de producto, sistemas de servicio, sistemas empresariales, sistemas de sistemas, sistemas de salud.
d. **Disciplinas relacionadas**: Ingeniería de software, Gestión de proyectos, Ingeniería industrial, Ingenierías especializadas.

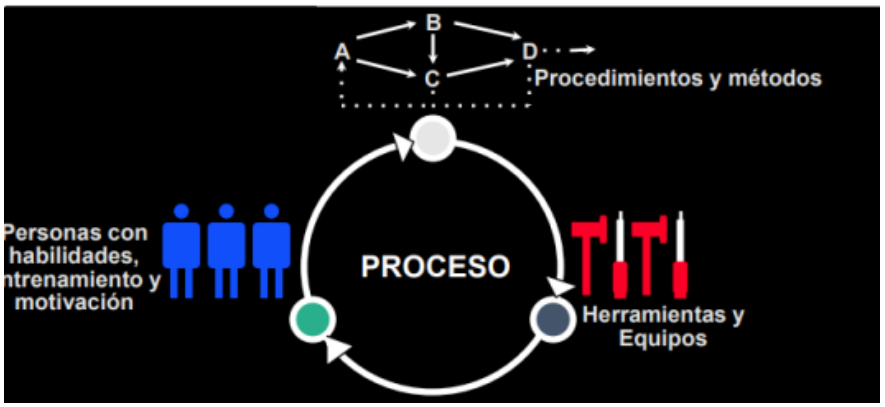
El proceso de software

Un proceso de software es un conjunto estructurado de actividades destinadas a desarrollar o mantener un sistema.



Cada organización adapta sus procesos según el tipo de producto, pero todo proceso debe estar explícitamente definido para poder administrarse.

- ➢ Un proceso es una secuencia de pasos ejecutados para un propósito determinado. (IEEE)
- ➢ Un proceso de software es un conjunto de actividades, métodos, prácticas y transformaciones utilizadas para desarrollar o mantener software y sus productos asociados. (SW-CMM)



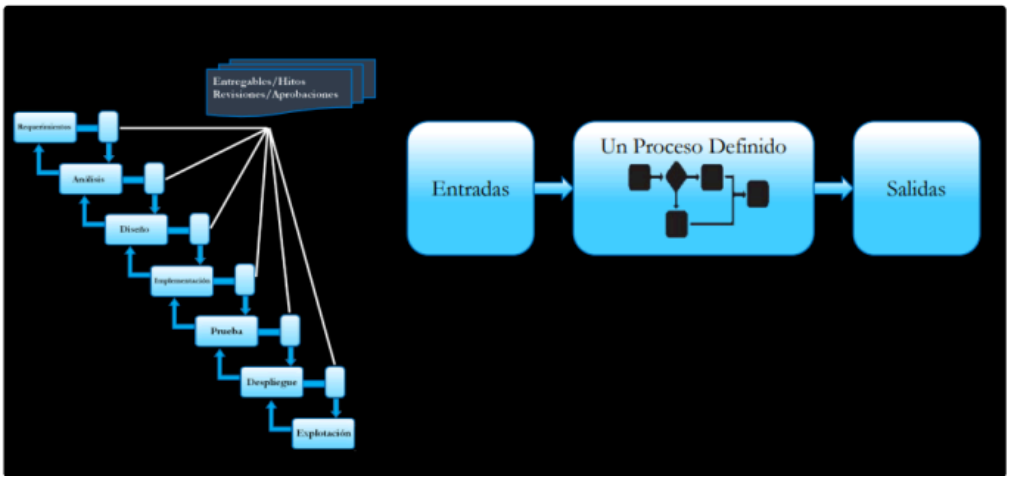
Procesos definidos y procesos empíricos

Una idea utilizada para explicar esta diferencia proviene de un experimento de la Universidad de California en Irvine: primero observaron por dónde caminaba naturalmente la gente sobre el césped y recién después construyeron las sendas.
Ese enfoque representa la lógica de un proceso empírico.

Actualmente, la industria del software se divide en dos grandes enfoques para la gestión del trabajo: los procesos definidos y los procesos empíricos. Tanto el pensamiento Ágil como el Lean están fuertemente alineados con los procesos empíricos.
Hoy en día, el framework más popular es Scrum, que se basa en la filosofía Ágil, mientras que Kanban tiene sus raíces en la filosofía Lean. Ágil no es una metodología ni un proceso; es un encuadre filosófico, un conjunto de ideas. Scrum no es lo mismo que Ágil, sino un marco de trabajo (framework). No explica paso a paso cómo programar software, sino cómo gestionar el proyecto.

- **Proceso definido**

Asume que repetir exactamente el mismo proceso produce resultados predecibles. Su administración depende de la planificación previa y del cumplimiento del procedimiento establecido.



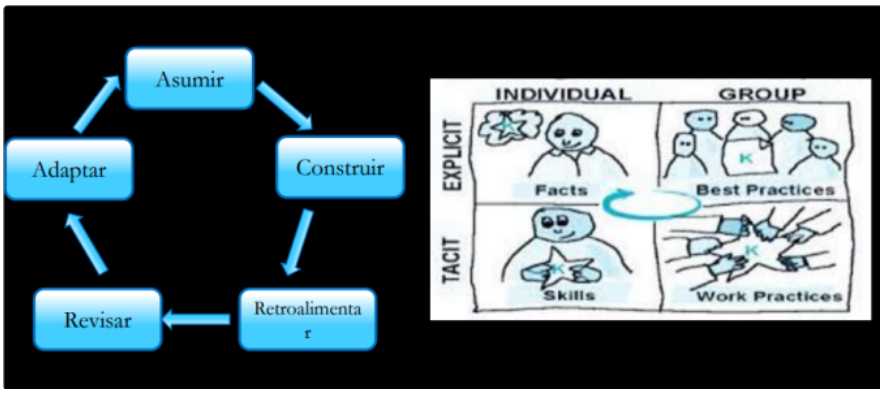
Entre los procesos definidos más conocidos se encuentran el PUD y el RUP. Su dinámica de trabajo consiste en intentar abarcar todo lo necesario para construir software desde el inicio. Para manejar la enorme complejidad, dividen el trabajo en fases, que a su vez se agrupan en actividades específicas. Cuando se inicia un proyecto bajo un proceso definido, la organización debe adaptar ese gran "manual" al proyecto específico, decidiendo qué partes aplicar y cuáles omitir. Su objetivo principal es lograr la repetibilidad del éxito.

- **Proceso empírico**

Parte del supuesto de que existen muchas variables cambiantes, por lo que repetir el proceso no garantiza obtener exactamente el mismo resultado. El control se logra mediante tres mecanismos:

- transparencia
- inspección frecuente
- adaptación continua

Este enfoque resulta especialmente adecuado para actividades creativas y complejas como el desarrollo de software.



Por el contrario, los procesos empíricos asumen que la experiencia pasada no se puede extrapolar directamente a nuevos proyectos debido a la gran cantidad de variables cambiantes, aunque sí suma a la experiencia individual. Nacen de la necesidad de empoderar a los equipos, buscando que estén capacitados, motivados y autogestionados para definir su propia forma de trabajo.
El empirismo se sostiene sobre tres pilares fundamentales (claves en Ágil y Lean) que permiten generar experiencia in situ:

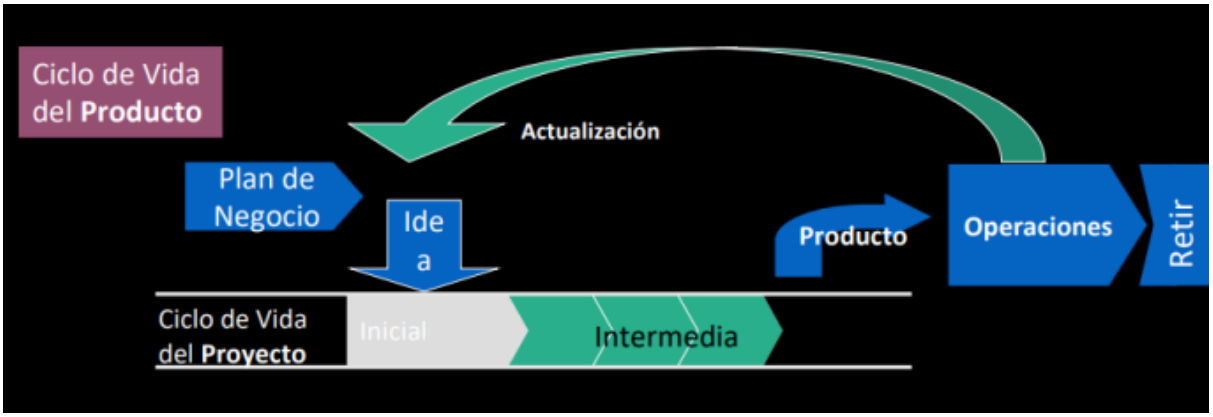
- Inspección: Consiste en ganar experiencia lo más rápido posible. Se apoya fuertemente en los ciclos de vida iterativos, ya que los periodos de trabajo cortos permiten frenar, revisar y aprender mediante una retroalimentación temprana y frecuente.
- Adaptación: En base a lo aprendido durante la inspección, se define qué ajustes son necesarios para mejorar. Estos cambios pueden aplicarse sobre el producto, el proceso, las personas o el proyecto en general.
- Transparencia: Es una actitud indispensable. Significa ser honesto, solidario y entender que "nadie se salva solo". Si se oculta información o se miente, no hay transparencia y se destruye la posibilidad de inspeccionar y adaptar correctamente.

No tiene sentido intentar mezclar Scrum con un modelo predictivo tradicional (Cascada), porque se rompe el ciclo de retroalimentación temprana y, sin él, es imposible generar la experiencia empírica necesaria.

Ciclos de vida

El ciclo de vida es la representación gráfica de la serie de pasos y fases a través de los cuales progresa el trabajo. Tanto el producto general como los proyectos individuales poseen sus propios ciclos.

- **Ciclo de vida del producto**: describe toda la existencia del sistema.
- **Ciclo de vida del proyecto**: describe únicamente el trabajo necesario para construir o modificar ese producto.



Modelos de ciclo de vida del proyecto

Los modelos de ciclo especifican cuáles son las fases del proceso (como requerimientos, diseño o pruebas) y en qué orden se llevarán a cabo. Se clasifican fundamentalmente en secuenciales, iterativos y recursivos.

Los modelos se clasifican en:

- ➢ Secuenciales: donde cada fase precede a la siguiente.
- ➢ Iterativos e incrementales: donde el producto evoluciona mediante sucesivas iteraciones.
- ➢ Recursivos: donde el proceso vuelve repetidamente sobre sí mismo para refinar la solución.

Software en contexto

El software no es solo código, sino un conjunto integral que abarca programas, procedimientos, reglas, documentación y datos.



En esencia, el software es información estructurada con propiedades lógicas y funcionales. Esta información es creada y mantenida en varias formas y representaciones, siendo finalmente confeccionada para ser procesada por una computadora en su estado más desarrollado.

Cambios en el software

El software está en constante evolución. Las modificaciones tienen su origen en diversas situaciones, tales como los cambios del negocio y nuevos requerimientos, el soporte de modificaciones en productos asociados, o la reorganización de las prioridades de la empresa debido a su crecimiento. Además, factores como ajustes en el presupuesto, defectos encontrados que deben corregirse y nuevas oportunidades de mejora impulsan esta evolución continua.

Disciplinas de soporte del Software

Para sostener la calidad del producto y del proceso a lo largo de los cambios, existen disciplinas de soporte fundamentales:

- Aseguramiento de Calidad de Software (QA): Enfocado en prevenir defectos.
- Administración de Configuración de Software (SCM): Gestiona la evolución de los artefactos.
- Control de Calidad (Proceso y Producto): Y las pruebas de software (Testing).

La SCM es una actividad “paraguas”, lo que significa que es transversal a todo el proyecto y relevante para el producto a lo largo de todo su ciclo de vida.

Gestión de Configuración de Software (SCM)

La Gestión de Configuración tiene su origen a mediados de la década de 1950. Originalmente, se utilizaba para el desarrollo de hardware y control de producción, y luego fue adaptada al desarrollo de software. La definición formal (ANSI/IEEE 828, 1990) establece que es una disciplina que aplica dirección y monitoreo administrativo y técnico para identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar sus cambios, registrar el estado de implementación y verificar la correspondencia con los requerimientos.

¿Por qué gestionar la configuración?

El propósito principal de la SCM es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica poder identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su origen intacto.

➢ Integridad del producto

Un producto íntegro es aquel que satisface plenamente las necesidades del usuario, puede ser fácil y completamente rastreado durante todo su ciclo de vida, satisface los criterios de performance exigidos y cumple con las expectativas de costo pautadas.

➢ Problemas comunes sin SCM

Si no se manejan correctamente los componentes, los equipos se enfrentan a graves problemas, como la pérdida de componentes o de cambios (trabajar con un archivo desactualizado), la falta de sincronía entre el código fuente y el ejecutable, la regresión de fallas ya resueltas, el doble mantenimiento, la superposición de modificaciones por distintos desarrolladores y la implementación de cambios no validados.

Conceptos clave de SCM

- Ítem de Configuración de Software (SCI / IC)

Se llama ítem de configuración (IC) a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo, y sobre los cuales necesitamos conocer su estado y evolución.

Ejemplos de IC incluyen: documentos de diseño, código fuente, ejecutables, planes de proyecto, propuestas de cambio, prototipos de interfaz, manuales de usuario, requerimientos, arquitectura, estándares de codificación, casos de prueba, gráficos e incluso el registro del proyecto.

- Evolución de los IC: Versiones y Variantes

La evolución de estos ítems se gestiona mediante el control de versiones y se puede representar gráficamente en forma de grafo.

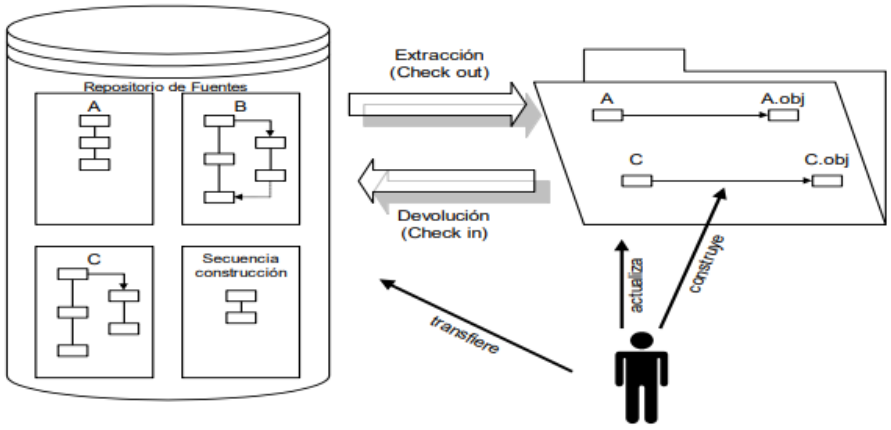
- Versión: Desde el punto de vista de la evolución, es la forma particular de un artefacto en un instante o contexto dado. El control de versiones sigue la pista de un IC a lo largo del tiempo.
- Variante: Es una versión de un IC (o de toda la configuración) que evoluciona por separado para representar configuraciones alternativas. Por ejemplo, un software

puede adoptar distintas formas dependiendo del sistema operativo donde se instale o de funciones opcionales

La Configuración del Software es, por tanto, un conjunto de ítems de configuración con su correspondiente versión ensamblados en un momento determinado.

- Repositorio

Un repositorio es una base de datos (o varias) de información que contiene los ítems de configuración. Su función es mantener la historia de cada IC junto con sus atributos y relaciones.



Tipos de repositorios:

1. Repositorios Centralizados: Un único servidor contiene todos los archivos con sus versiones. Los administradores tienen un gran control sobre el repositorio, pero si el servidor falla, el equipo queda paralizado
2. Repositorios Descentralizados: Cada cliente tiene una copia exactamente igual del repositorio completo en su máquina. Si un servidor falla, es fácil recuperarlo copiando de un cliente. Además, posibilita flujos de trabajo (workflows) que no están disponibles en el modelo centralizado.

- Línea Base (Baseline)

Una línea base es una configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo. Se utilizan etiquetas (tags) para marcar estas baselines y no deben confundirse con la simple versión del producto. Sirven como base firme para desarrollos posteriores y sólo pueden modificarse a través de un procedimiento formal de control de cambios. Su gran utilidad es permitir viajar atrás en el tiempo y reproducir exactamente el entorno de desarrollo en un momento dado del proyecto. Las líneas base pueden ser de especificación (requerimientos, diseño) o de productos que ya han superado un control de calidad.

- Ramas (Branches)

- Creación de ramas: Existe siempre una rama principal (trunk o master). Las ramas secundarias sirven para bifurcar el desarrollo y permitir la experimentación aislada. Generalmente se crean con un propósito semántico claro (ej. corregir un bug o crear una feature).

- Integración de ramas: La operación de unir ramas se llama merge. Esto lleva los cambios aislados hacia la rama principal. Durante este proceso pueden surgir conflictos que se resuelven utilizando herramientas de diff. Eventualmente, todas las ramas deberían integrarse a la principal o ser descartadas.

Actividades Fundamentales de la SCM

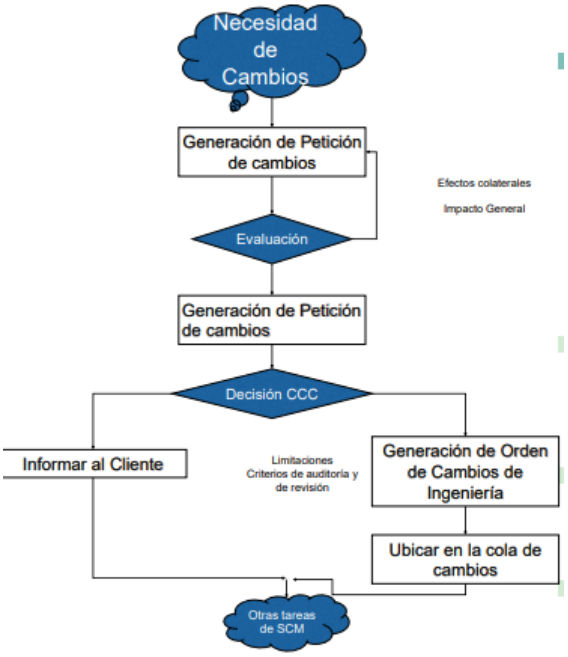
El proceso de gestión de la configuración se sostiene sobre cuatro pilares operativos clave.

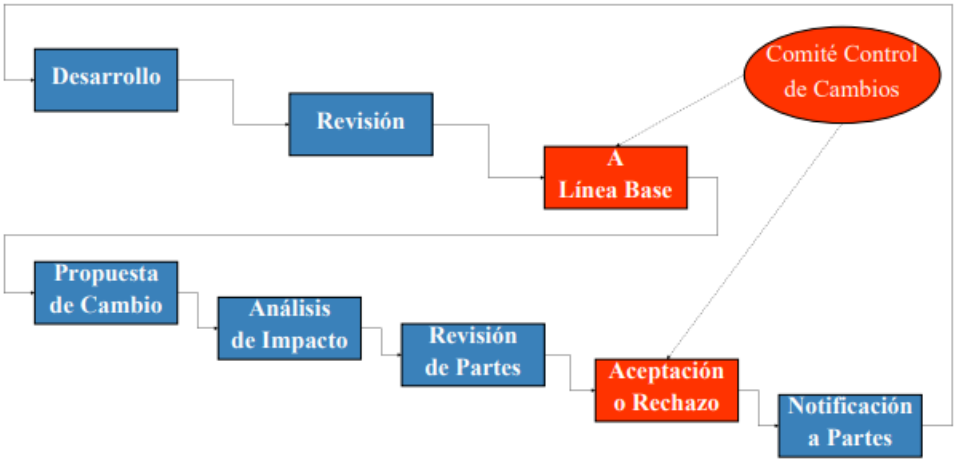
1. Identificación de Ítems

Consiste en la identificación unívoca de cada ítem de configuración. Requiere establecer convenciones y reglas de nombrado estrictas, y definir una estructura clara del repositorio para ubicar rápidamente cada artefacto.

2. Control de Cambios

Todo cambio tiene su origen en un Requerimiento de Cambio aplicado a ítems que ya se encuentran bajo una línea base. Es un procedimiento formal que involucra a diferentes actores y exige una evaluación del impacto antes de proceder.



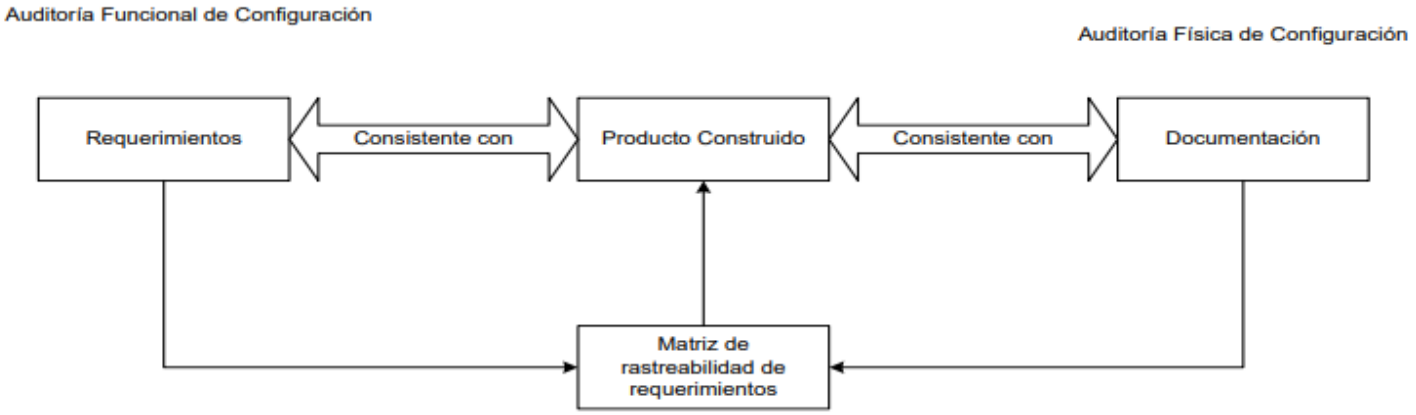


El Comité de Control de Cambios (CCB): Es el grupo encargado de aprobar o rechazar los requerimientos. Está formado por representantes de todas las áreas involucradas: Análisis, Diseño, Implementación, Testing y otros stakeholders

3. Auditorías de Configuración

Sirven a los procesos básicos de validación (asegurar que se construya el producto correcto para el usuario) y verificación (asegurar que el producto cumple con los objetivos técnicos preestablecidos).

- Auditoría física de configuración (PCA): Asegura que lo que está documentado para cada IC en la línea base realmente se haya construido o alcanzado físicamente.
- Auditoría funcional de configuración (FCA): Es una evaluación independiente que controla que la funcionalidad y la performance reales de cada ítem sean totalmente consistentes con la especificación de requerimientos.



4. Informes de Estado

Se ocupa de mantener los registros vivos de la evolución del sistema. Cómo maneja un gran volumen de información, suele implementarse mediante procesos automáticos. Incluye reportes de rastreabilidad (traceability) de todos los cambios realizados. Debe poder responder preguntas críticas: ¿Cuál es el estado de un ítem? ¿Se aprobó el requerimiento de cambio? ¿Qué versión exacta contiene la mejora solicitada?

Plan de Gestión de Configuración

Todo proyecto debe formalizar sus reglas en un plan que contenga:

- Reglas de nombrado de los IC.
- Herramientas a utilizar para SCM.
- Roles e integrantes del Comité de Cambios.
- El procedimiento formal para solicitar y aplicar modificaciones.
- Plantillas de formularios y procesos de auditoría.

SCM en Ambientes Ágiles

En los enfoques iterativos, la gestión de configuración cambia su filosofía de control estricto hacia la facilitación continua.

- ➢ Sirve al equipo de desarrollo: En Agile, el SCM ayuda a los practicantes y coordina el trabajo, en lugar de intentar controlar o frenar a los desarrolladores.
- ➢ Abraza el cambio: Responde rápidamente a las modificaciones en lugar de intentar evitarlas con burocracia.
- ➢ Transparencia extrema: Busca ser "sin fricción", impulsando la automatización de integraciones frecuentes y rápidas.

➢ Enfoque Lean: Promueve la trazabilidad y la documentación útil, eliminando todo el desperdicio. Ofrece feedback continuo y visible sobre la calidad, estabilidad e integridad del sistema

Consejos para aplicar SCM en Agile

- Es una responsabilidad compartida por todo el equipo, no de un administrador aislado.
- Hay que automatizar tanto como sea posible (integraciones, pruebas, despliegues).
- El equipo debe estar educado en buenas prácticas de versionado.
- Las tareas de SCM deben estar embebidas y naturalizadas dentro de las tareas diarias necesarias para alcanzar el objetivo del Sprint.

Gestión de Proyectos en Software

En el contexto del desarrollo, suele diferenciarse el proceso, el proyecto, el producto y las personas.
En esta materia, el producto final es el software en sí, pero nuestro foco de estudio principal está puesto en el proyecto y su gestión.
Las principales metodologías y filosofías de trabajo actuales se basan en los pensamientos Ágil y Lean. Estas corrientes destacan por encima de todo el factor humano, afirmando que la participación e interacción de la gente es fundamental para lograr un buen producto. La base fundamental para que una iniciativa sea considerada un proyecto es que el resultado obtenido sea único. Además, debe contar obligatoriamente con una fecha de inicio y una fecha de fin, desarrollándose y cumpliéndose de manera gradual.

El Manifiesto Ágil

Como dato histórico, la filosofía ágil fue creada por desarrolladores frustrados ante el constante fracaso de sus proyectos de software tradicionales. De esa crisis surgió el famoso Manifiesto Ágil.

- Valores ágiles

Los pilares que sostienen el comportamiento de los equipos son:

- Individuos e interacciones sobre procesos y herramientas.
- Software funcionando sobre documentación extensiva.
- Colaborar con el cliente sobre la negociación contractual.
- Responder al cambio sobre seguir un plan estricto.

- Los 12 principios del Manifiesto Ágil

1. Satisfacción del cliente: Priorizar la entrega temprana y continua de software con valor.
2. Aceptar el cambio: Dar la bienvenida a nuevos requisitos, incluso en etapas tardías del desarrollo.
3. Entregar con frecuencia: Publicar software funcional de forma regular (en ciclos de pocas semanas a meses).
4. Trabajo en conjunto: Colaborar a diario entre los expertos del negocio y los desarrolladores.
5. Individuos motivados: Construir los proyectos en torno a personas a las que se les brinde apoyo y confianza.
6. Comunicación cara a cara: Conversar de forma directa y presencial para maximizar la eficiencia de la información.
7. Software funcional: Utilizar el software que realmente funciona como la métrica principal de progreso.

8. Desarrollo sostenible: Mantener un ritmo de trabajo constante que pueda sostenerse indefinidamente.
9. Excelencia técnica: Prestar atención continua al buen diseño y a la calidad técnica.
10. Simplicidad: Fomentar el arte de maximizar la cantidad de trabajo no realizado (evitar complejidades innecesarias).
11. Equipos auto-organizados: Confiar en que las mejores arquitecturas y diseños emergen de equipos con autonomía.
12. Mejora continua: Reflexionar periódicamente como equipo para ajustar y perfeccionar el rendimiento.

- ¿Qué significa ser Ágil?

Tal como se mencionó, Ágil no es un proceso estricto. Es un compromiso y un balance útil entre no tener nada de proceso y tener demasiado proceso. Los equipos ágiles valoran la planificación continua a múltiples niveles y realizan entregas frecuentes, iterativas y priorizadas, siempre respaldadas por prácticas técnicas disciplinadas (como integración continua y testing concurrente). La principal diferencia con el desarrollo clásico no es solo escribir menos documentación, sino que los métodos ágiles son adaptables en lugar de predictivos y se orientan plenamente a las personas en lugar de a los procedimientos

- Mitos de la agilidad

Ser ágil no es ser indisciplinado:

- Agile deficiente: Trabajar sin plan y sin arquitectura bajo la excusa de ser "ágiles".
- Agile eficaz: Contar con un plan y una arquitectura generales, pero diseñados para ser maleables y adaptables.
- Proyectos tradicionales: Implican una planificación y arquitectura exhaustivas y rígidas desde el día cero.

Requerimientos ágiles

El ciclo de un requerimiento nace para cubrir un problema, necesidad u oportunidad y culmina entregando una solución de valor. La clave de la agilidad está en usar el "valor" como brújula para construir el producto correcto, apoyándose en historias para ilustrar qué se debe construir y descubriendo sobre la marcha qué significa hacer "solo lo suficiente". Esto contrasta fuertemente con el modelo tradicional (BRUF - Big Requirements Up Front), donde se intenta definir absolutamente todo de antemano, lo que suele resultar muy costoso y generar un desperdicio significativo de esfuerzo.

➢ Gestión iterativa

En entornos ágiles, los requerimientos cambiantes no son un problema, sino una ventaja competitiva si el equipo tiene la capacidad de reaccionar. Cada iteración se encarga de implementar los requerimientos que actualmente tienen la prioridad más alta. Si surge una nueva necesidad, simplemente se estima, prioriza y agrega a la pila de trabajo (Backlog). Del mismo modo, cualquier requerimiento puede ser repriorizado o incluso eliminado en cualquier etapa.
Definición de Valor: Es la obtención de un beneficio (tangible o intangible). Se asocia directamente a la utilidad o satisfacción que se le ofrece a los usuarios finales por cada funcionalidad completa que se entrega.

➢ La riqueza de la comunicación

El medio por excelencia es el cara a cara, ya que permite que fluya información vocal, subvocal y gestual, posibilitando una realimentación instantánea. La efectividad varía: el medio más "cálido" (efectivo) son dos personas debatiendo frente a un pizarrón; luego baja al teléfono, chat, videollamada, papel, hasta llegar al medio más "frío", que son las grabaciones de audio o documentos estáticos.

➢ Tipos de Requerimientos

- Requerimiento de Negocio: Objetivos de alto nivel, ej. "Disminuir un 20% el tiempo en atención al cliente".
- Requerimiento de Usuario: Lo que necesita hacer la persona, ej. "Consultar el estado de cuenta en línea".
- Requerimiento Funcional: Qué debe hacer el sistema, ej. "Generar reporte de saldos en formato PDF".
- Requerimiento No Funcional: Restricciones o atributos de calidad, ej. "Cumplir normas de seguridad bancaria".
- Requerimiento de Implementación: Decisiones operativas, ej. "Alojar la base de datos en servidores de la nube".

Los requerimientos de negocio y usuario conforman el dominio del problema, mientras que los de software estructuran el dominio de la solución.

➢ Tradicional vs Ágil

- Tradicional (dirigido por el plan): Los requisitos son fijos. Los recursos y el tiempo son variables estimadas.
- Ágil (dirigido por valor): Los recursos y el tiempo son fijos (iteraciones de tiempo establecido). El alcance (los requisitos) es la variable estimada.

Para no olvidar: Los cambios son la única constante. Siempre se cumple la máxima de que "el usuario sabe lo que quiere recién cuando recibe lo que pidió". El objetivo final no es cumplir con un papel lleno de especificaciones, sino entregar una solución de valor.

➢ Comparativa de principios

Principios Ágiles	Principios Lean
1- Satisfacer al cliente mediante releases tempranos y frecuentes (2-4 semanas)	1- Eliminar el desperdicio
2- Recibir cambios de requerimientos, aun en etapas finales	4- Diferir Compromisos
4- Técnicos y expertos del negocio trabajando juntos todo el proyecto	6- Ver el todo
6- El medio de comunicación por excelencia es el cara a cara	
11- Mejores arquitecturas surgen de equipos auto-organizados	

Historias de Usuario (User Stories)

"Se las llama historias porque se supone que usted cuenta una historia."

Lo que se escribe físicamente en una tarjeta o ticket no es el requerimiento final; lo verdaderamente crítico es la conversación que esa tarjeta genera. Como explica la bibliografía, decidir precisamente

qué construir es la parte más difícil del software. Es un trabajo conceptual que, si se especifica de manera incorrecta y rígida desde el inicio, es doloroso rectificar más adelante.
Una historia de usuario es una descripción breve e informal de una funcionalidad, contada siempre desde la perspectiva de quien la va a utilizar. Sin embargo, el texto escrito no reemplaza la comunicación real; la historia se compone de tres partes inseparables: la tarjeta (el texto), la conversación (el debate con el equipo) y la confirmación (los criterios de aceptación).
Regla de redacción: Cada historia de usuario tiene un valor de negocio ÚNICO. Si al redactar la historia se utiliza un "y para..." (ej. "para llevar un control y para exportar a Excel"), es un claro indicativo de que se están mezclando valores y la historia debe dividirse en dos.

➢ ¿Qué es y para qué sirve?

Las User Stories son un formato multipropósito que representa una necesidad del usuario, describe una porción del producto y funciona como unidad de planificación. Actúan como un token (una ficha) que nos recuerda que debemos tener una conversación y, al mismo tiempo, son un mecanismo para diferir dicha conversación hasta que sea realmente el momento de construir la funcionalidad. Las historias de prioridad alta se modelan con mucho detalle porque entrarán en la iteración inmediata. Las de prioridad baja se mantienen abstractas, ya que podrían cambiar o ser removidas del Backlog por el Product Owner antes de llegar a ellas.

➢ Anatomía de una Historia (Las 3 C)

Una historia consta de tres partes inseparables:

1. Tarjeta (Card): El texto físico o digital.
2. Conversación: El debate entre el negocio y el equipo técnico.
3. Confirmación (Criterios de aceptación): Cómo sabemos que está terminada

El estándar más utilizado para redactar la tarjeta es: Como <rol de usuario>, yo quiero <actividad / acción> para poder <valor de negocio o beneficio>. Ejemplo: Como Conductor, quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

➢ Diferentes niveles de abstracción

- Épica: Una historia de usuario masiva, demasiado grande para entrar en un solo ciclo y que debe dividirse.
- Tema: Un conjunto de historias de usuario que están lógicamente relacionadas.
- Historia de Usuario: Una porción pequeña que describe funcionalidad desde la perspectiva del usuario

Las historias de usuario deben representar porciones verticales del sistema, es decir, incluir suficientes cambios cruzando todas las capas arquitectónicas (BD, Backend, Frontend) para entregar un incremento real de valor. No deben ser cortes horizontales (ej. "Hacer solo las tablas de la base de datos").

Entendiendo al Usuario

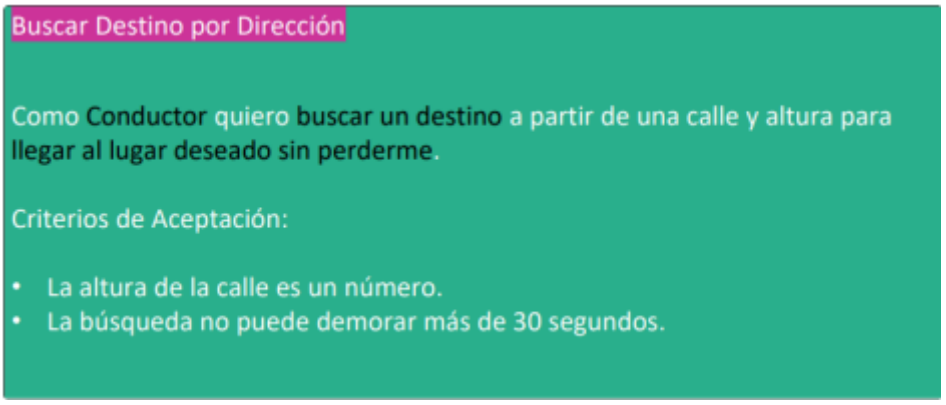
Para redactar buenas historias, hay que conocer a quién las usa:

- Tarjeta de Rol de Usuario: Describe rápidamente el nivel técnico, frecuencia de uso y metas de un usuario tipo.
- Personas (Arquetipos): Creación de perfiles ficticios pero detallados (con nombre, contexto, familia, trabajo) para generar empatía (ej. "Mario de 35 años, teletrabaja, experto en PC").

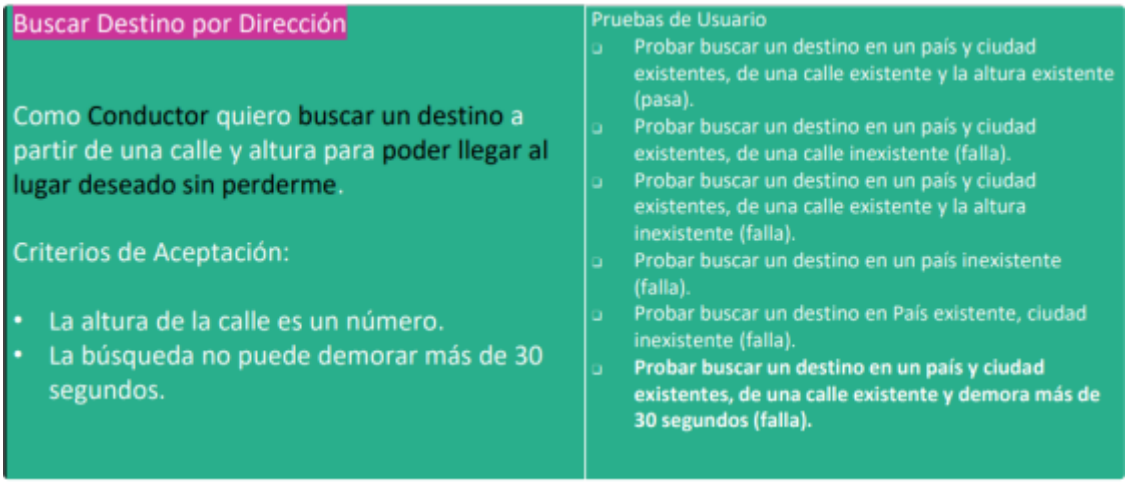
- Personajes Extremos: Pensar la interfaz para casos atípicos (ej. diseñar una agenda digital para el Papa vs. un traficante de drogas) obliga al equipo a contemplar fuertes requerimientos de visibilidad y seguridad/privacidad.
- Usuarios Representantes (Proxies): Apoyarse en gerentes, vendedores o soporte cuando el cliente final no está disponible. Sin embargo, nunca son tan ideales como hablar con los usuarios reales.

Criterios de Aceptación (La confirmación)

Definen los límites de la historia y funcionan como los requerimientos funcionales mínimos para considerar que aporta valor. Ayudan a que todo el equipo comparta la misma visión, guían el diseño de las pruebas y, lo más importante, le avisan a los desarrolladores en qué momento exacto parar de programar. Un buen criterio define la intención, no la solución técnica. Debe mantener un nivel relativamente alto (ej. "El usuario debe elegir al menos una cuenta para operar"). Los micrométricos detalles visuales o técnicos (como el formato específico del salto a 999.999, o usar una lista desplegable en vez de un checkbox) surgen de las charlas y se bajan a la documentación interna o directamente a las pruebas de aceptación automatizadas.



El trabajo con estas pruebas es un proceso en dos pasos: primero se escriben ideas o condiciones al dorso de la tarjeta (o en el ticket virtual) y luego, al momento de programar, se diseñan las pruebas de validación completas.



El Modelo INVEST y definiciones de estado

Para asegurar que una historia tiene calidad, debe cumplir con el acrónimo INVEST:

- Independent: Se pueden calendarizar e implementar en cualquier orden.
- Negotiable: Expresan el qué, permitiendo debatir el cómo.
- Valuable: Proveen valor medible para el cliente.
- Estimatable: El equipo las entiende lo suficiente como para asignarles un costo.

- Small: Su tamaño permite completarlas íntegramente dentro de un mismo ciclo (Sprint).
- Testable: Sus criterios permiten demostrar sin lugar a duda que fueron completadas.

Acompañando a INVEST, los equipos definen dos acuerdos:
- Definition of Ready (DoR): Requisitos que debe tener la historia para entrar a desarrollo
- Definition of Done (DoD): Lista de validaciones (testing, código limpio, documentación) para darla por terminada.

➢ Olores en las Historias (Malas prácticas)

Una historia está mal redactada si se parece a una especificación técnica o limita la solución tecnológica. Ejemplos de "malos olores":
- "El usuario ejecutará el sistema en Windows o Linux" (Restricción de infraestructura encubierta).
- "Se usará Log4J para registrar los errores en un archivo" (Decisión puramente de arquitectura interna, no hay valor de usuario final).
- Mezclar lógicas de programación o campos detallados de base de datos dentro del requerimiento funcional.

Importante: Las User Stories no son casos de uso exhaustivos. Necesitan nulo mantenimiento y suelen descartarse (o cerrarse) una vez implementadas. La verdadera documentación evoluciona iterativamente a partir del código terminado.

Spikes

Un Spike es un tipo especial de historia de usuario que se crea pura y exclusivamente para quitar riesgo e incertidumbre del proyecto. Se utiliza cuando el equipo debe invertir tiempo investigando antes de poder estimar o programar una funcionalidad real. Se dividen en dos enfoques:
- Spikes Técnicos: Investigar si usar cierta librería, evaluar un cuello de botella en rendimiento, o decidir si "hacer o comprar" una herramienta. Ayudan a asegurar la viabilidad tecnológica.
- Spikes Funcionales: Analizar incertidumbres del usuario. Se resuelven armando mockups o prototipos desechables para recolectar feedback rápido de los interesados (ej. definir cómo quieren ver realmente un gráfico de métricas complejo).

Si un equipo necesita graficar consumo eléctrico, podrían requerirse ambos: un Spike Técnico para ver si los sensores aguantan la tasa de actualización (push vs pull), y un Spike Funcional para prototipar la interfaz con el cliente.

➢ Reglas para los Spikes

Deben ser estimables, demostrables y aceptables (al igual que una historia, su investigación debe cerrarse y presentarse). Sin embargo, su uso debe ser la excepción, no la regla. Las incógnitas de una historia normal se resuelven durante el sprint; los Spikes solo se reservan para obstáculos graves y estructurales.

Máxima ágil sobre el requerimiento: Diferir el análisis detallado tan tarde como sea posible (es decir, justo antes de empezar a programar). Hasta ese momento, mantenemos las ideas atrapadas en Historias de Usuario flexibles, asumiendo que las User Stories no son requerimientos de software tradicionales, ni exhaustivos.

Estimaciones de software:

El pensamiento ágil y el empirismo surgen en contraposición a la gestión tradicional de proyectos para resolver problemas no cubiertos por los procesos definidos. Por ello, para entender la estimación ágil, primero debemos comprender su base conceptual en la gestión tradicional.

Estimaciones: tiene que ver con la predicción acerca de algo que podría pasar en el futuro. La dificultad de esto es el nivel de incertidumbre con el que trabajamos para definir esa predicción, para esto debemos buscar métodos y técnicas para que esas predicciones sean lo más efectivas posibles.

- **Regla de oro**: Las estimaciones no son el plan y no son compromisos. Son herramientas y entradas de información para determinar factibilidad y establecer compromisos de negocio. El plan es una cosa y la estimación es otra.
- Las estimaciones NO son precisas.
- Estimar no es planear y planear no es estimar.
- Las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
- Mientras mayor sea la diferencia entre lo estimado y lo planeado, aumenta el riesgo.
- Las estimaciones no son compromisos.

El nivel de incertidumbre en las estimaciones va disminuyendo a medida que el proyecto avanza, pero al ocurrir eso el proceso de estimaciones es algo que hacemos a lo largo de toda la vida del proyecto y tomando decisiones en función de eso.

Posibles errores al estimar

Dado que se trabaja con incertidumbre, pueden surgir errores por múltiples factores:

- Usar una técnica incorrecta.
- No ir ajustando los valores conforme a la realidad empírica del proyecto.
- Utilizar un solo método, cuando lo ideal en entornos inciertos es combinar más de una técnica de estimación.
- Basarse en datos históricos que no aplican al contexto actual.

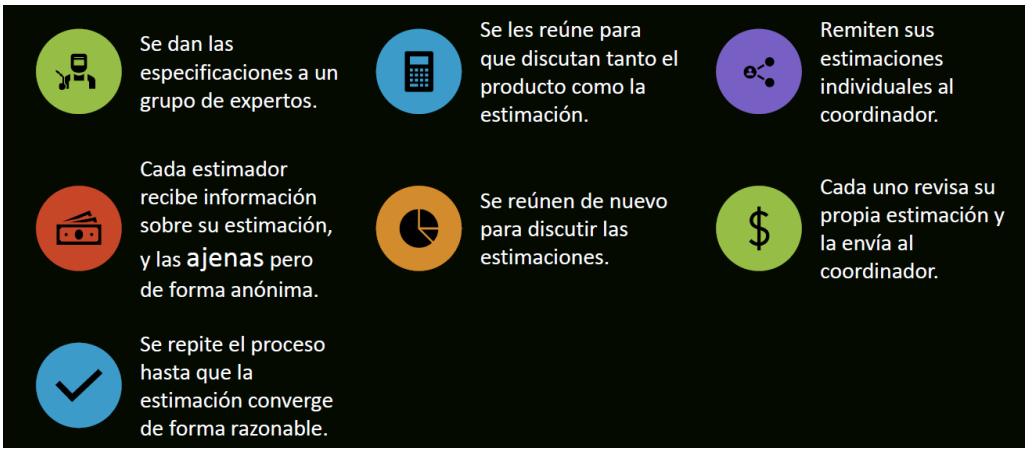
Técnica fundamental de estimación:

- **Contar**: requerimientos, personas involucradas, funcionalidades, horas de trabajo, cantidad de días, etc.

Métodos de estimación tradicionales: en la gestión tradicional, las estimaciones suelen usar **medidas absolutas** (como horas lineales), las cuales tienen un significado universal por sí mismas. En este contexto destacan los métodos basados en la experiencia.

- **Datos históricos**: tiene que ver con utilizar información existente para poder realizar estimaciones relacionadas con un nuevo proyecto.
  Dificultad: no siempre la experiencia es extrapolable. Incluso si dos proyectos parecen similares, cambian los equipos de trabajo, el contexto y las tecnologías. Además, exige que la organización tenga una cultura estricta de almacenamiento para poder buscar y comparar proyectos tipificados correctamente.
- **Juicio de experto**:
  - ★ Puro: tengo a un experto que sabe del tema, que estudia las especificaciones del proyecto y a partir de esto realiza la estimación.
    Enfoque más utilizado en la práctica.
    Dificultad:
    - Si el experto desaparece, la empresa deja de estimar.
    - Subjetividad por parte del experto.
    - Que el experto considere que tiene ciertos derechos que en realidad no y por eso estime mal.

★ Delphi: un grupo de personas son informadas y tratan de adivinar lo que costará el desarrollo tanto en esfuerzo, como en duración. Las estimaciones en grupo suelen ser mejores que las individuales. El secreto es que la estimación se hace en términos de consenso:



★ Analogía.

Estimaciones en ambientes ágiles: a diferencia de las estimaciones tradicionales, las features/stories son estimadas usando una unidad de medida de tamaño relativo (o sea no absoluta) conocida como story points (SP). Este tipo de unidad de medida solo es válida para las personas que participan del proyecto y no está basada en el tiempo, sino más bien con el tamaño.

Estimación relativa:

- Las personas no saben estimar en términos absolutos.
- Somos buenos comparando cosas.
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
- Se emplea mejor el tiempo de análisis de las stories.
- En Agile, las historias se estiman usando Story Points (SP). No es una medida de tiempo, sino una unidad de medida relativa específica del equipo que indica el "peso" o tamaño de la historia.
- El tamaño (peso) de una funcionalidad se compone de tres variables inseparables:
  - ★ **Complejidad**: cuán intrincada es una feature/story.
  - ★ **Esfuerzo**: cuánto trabajo es requerido para hacer o completar una feature/story.
  - ★ **Incertidumbre**: cuántos riesgos o cosas desconocemos sobre la historia.

Descripción del Trabajo vs Esfuerzo: Las estimaciones basadas en tiempo son más propensas a errores debido a que pueden variar por muchas razones como:

- Habilidades.
- Conocimiento.
- Asunciones.
- Experiencia.
- Familiaridad con los dominios de aplicación/negocio.

El tamaño NO ES esfuerzo.
Para el tamaño, contamos con diferentes escalas:

- **Por números**: 1 a 10.
- **Talles de remeras**: S, M, L, XL.
- **Serie** 2^n: 1, 2, 4, 8, 16, 32, 64, etc.
- **Serie de Fibonacci**: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc. (la más utilizada ya que es muy útil para medir user stories ya que éstas crecen de manera exponencial, no lineal).

Una vez elegida la escala no se cambia ya que de hacerlo, cambiamos el metro patrón.

Story Point: es una unidad de medida RELATIVA específica (del equipo) de, complejidad, incertidumbre/riesgo y esfuerzo, es lo que "el kilo" a la unidad de nuestro sistema de medición de peso.

- Story point da idea del "peso" de cada story y decide cuán grande (compleja) es.
- La complejidad de una feature/story tiende a incrementarse exponencialmente.

Velocidad: es la métrica de progreso empírico de un equipo. Se calcula sumando los Story Points de todas las historias de usuario que fueron completadas al 100% durante la iteración (nunca se cuentan historias a medias). Esta velocidad real es la que termina corrigiendo los errores de estimación iniciales.

Poker estimation: variante/mejora del método juicio experto delphi que combina la opinión de expertos, la analogía y la desagregación.

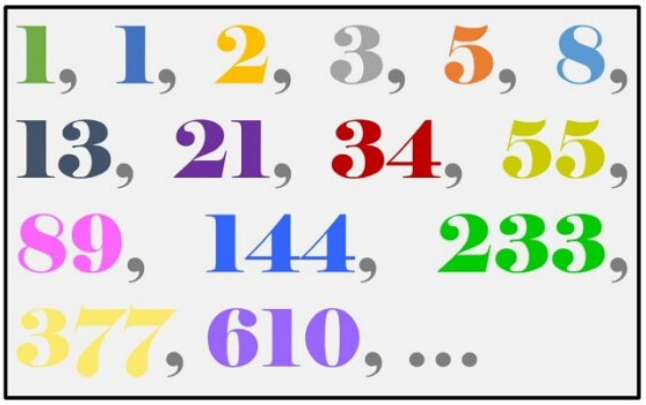
- **Regla fundamental**: las personas más competentes para estimar una tarea son pura y exclusivamente quienes van a desarrollarla.

Proceso de estimación (Paso a paso):
Cada desarrollador recibe un mazo de cartas físicas o virtuales con la secuencia de Fibonacci. El ciclo de estimación funciona así:

1. **Definir la Historia Canónica**: El equipo se pone de acuerdo y elige una historia muy pequeña y sencilla para asignarle un valor base de 1 (o a veces 2). Esta historia canónica no cambia y servirá como punto de comparación para todo el proyecto.
2. El Product Owner lee una nueva historia y el equipo hace todas las preguntas necesarias para despejar dudas.
3. Cada estimador hace un ejercicio mental de comparación: "¿Qué tan grande, compleja o riesgosa es esta historia comparada con nuestra historia canónica?" y selecciona una carta, poniéndola boca abajo en la mesa.
4. Se revelan todas las cartas al mismo tiempo (**Transparencia**).
5. Si todos coinciden, ese es el valor. Si hay dispersión, quienes pusieron los valores más altos y más bajos justifican sus motivos (**Inspección**). Alguien pudo ver un riesgo oculto o, por el contrario, notar que algo ya estaba resuelto.
6. Tras el debate, se vuelve a votar hasta lograr consenso (**Adaptación**).

¿Cómo "decodificar" las estimaciones?

Recordando el modelo INVEST, las historias deben ser pequeñas (Small) y estimables (Estimatable). Por ende, los números altos en las cartas son una señal de alerta de que la historia debe romperse:



- **0**: Hay demasiada incertidumbre. Falta investigación y la historia aún no está lista para ser estimada.
- **1, 2 o 3**: funcionalidades pequeñas a medianas. Es el tamaño ideal que buscamos.
- **5**: Funcionalidad media. Aún es totalmente manejable.
- **8**: Funcionalidad grande. Se puede hacer, pero el equipo ya debe preguntarse si no conviene dividirla. Debería ser el límite máximo aceptable.
- **13 o 20**: Es demasiado grande. Hay que cuestionar la razón de negocio y buscar obligatoriamente la forma de partirla.
- **40 o 100**: Imposible de realizar en un solo Sprint. Confirmación de que algo está muy mal concebido (es una Épica disfrazada).

Gestión de productos de software: cuando vamos a crear un nuevo producto de software, esa creación tiene una determinada línea de crecimiento.

1. Validación de hipótesis.
2. Definición de alcance.
3. Diseño y desarrollo.
4. Implementación.
5. Crecimiento y lealtad.
6. Experiencia del usuario.

El mayor riesgo no es fallar técnicamente, es construir a la perfección algo que nadie quiere (funcionalidades que no se utilizan).
El pensamiento Lean define la creación de valor como proveer beneficios reales a los clientes. Cualquier otra cosa es desperdicio.
La productividad no se mide en líneas de código, sino en averiguar sistemáticamente cuál es la cosa correcta a construir.

La verdadera productividad no se mide en líneas de código, sino en averiguar sistemáticamente cuál es la cosa correcta a construir. Escribir código no es sinónimo de crear valor. El éxito no se mide por la cantidad de funcionalidades entregadas, sino por la adopción real del cliente.

La brecha evolutiva del ingeniero de software:

- **Focalizado en tareas**: crear un producto de software que resuelva el problema que queremos resolver.
- **Focalizado en experiencias**: que ese producto de software le proporcione al usuario una experiencia amigable y que genere que quiera utilizar ese software y no otro.
- La transición exige dejar de pensar en "características del producto" para comenzar a diseñar "contextos de uso".

Producto Mínimo Viable (MVP): es un experimento diseñado para confrontar la realidad lo antes posible. Se construye a partir de una Propuesta de Valor Única (UVP), construyendo una pequeña versión del producto con el menor de los esfuerzos posibles para probar si la idea es apetecible para el mercado. Sin embargo, para validar esa hipótesis, necesitamos que nuestro usuario perciba el software como si este funcionara en su totalidad, aunque internamente sepamos que lo construimos ahorrando recursos.
Nos tenemos que asegurar de no demorarnos para no perder la oportunidad.

- **La audacia de cero**: el número cero invita a la pura imaginación de los inversores. Esto crea un incentivo brutal para retrasar el lanzamiento hasta que el producto sea "perfecto".
- **El costo del retraso**: posponer el MVP aumenta el esfuerzo desperdiciado, pierde feedback temprano esencial y maximiza el riesgo de construir algo que nadie quiere.

Propuesta de Valor Única (UVP): es la hipótesis principal. Lo que hace que el producto o servicio sea único en el mercado. A partir de esto creo el MVP que será el vehículo para probar la hipótesis. Debe incluir funciones básicas para ser genuinamente "viable". Luego, tengo que explorar si este MVP es aceptado y qué características señalan los usuarios que podríamos agregarle. Si todo el mundo identifica cosas diferentes, la hipótesis no se valida.

Pivote: a partir de lo que mostramos en el MVP, exploramos qué características piden los usuarios.

- Si todos los usuarios piden agregados totalmente distintos entre sí, la hipótesis no se valida: aún no hay mercado.
- Si todos piden la misma característica faltante, es momento de hacer un pivote.

Pivotar significa utilizar el aprendizaje obtenido para construir un segundo MVP diferente. Como usamos el mínimo esfuerzo inicial, descartar un MVP que no sirvió no representa un problema grave.



Escalando la Propuesta: El Ecosistema Funcional

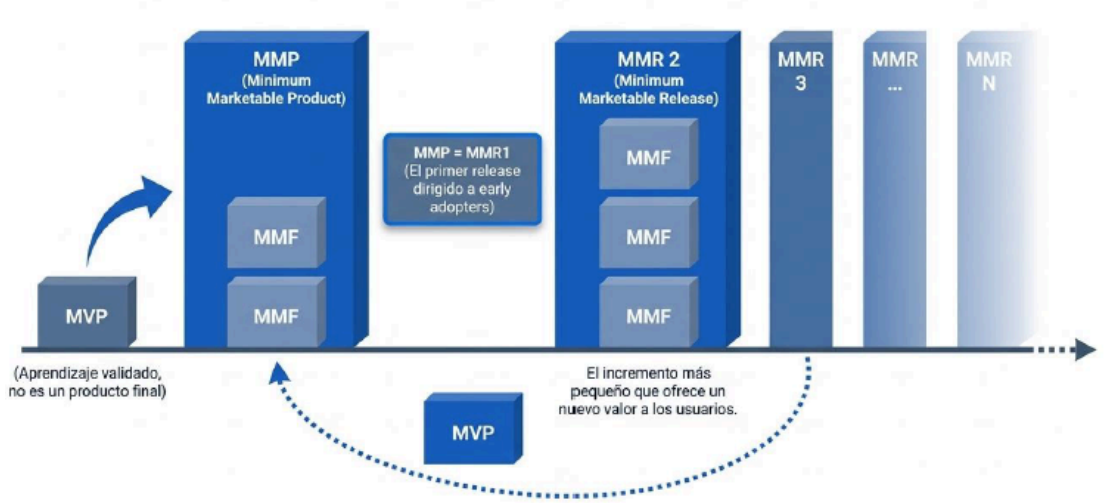
Suponiendo que ya llegamos al MVP que agrega valor al mercado, ya tenemos que empezar a pensar en el producto mínimo comercializable. En simultáneo, también puedo ir trabajando con características mínimas viables (dentro del producto que ya empieza a existir, puedo seguir haciendo pruebas de valor con estas pequeñas características).

El corte geológico (reduciendo el riesgo)
Aunque los MVP, MMF y MVF son atómicos desde la perspectiva de negocio (no se puede aprender de algo más pequeño), son gigantescos desde la perspectiva de desarrollo. Por ello, se abordan como rebanadas finas de ejecución técnica.

- **MMF (Minimum Marketable Feature)**: Son nodos sólidos con alta certeza comercial. Es la pieza más pequeña de funcionalidad que puede ser liberada aportando valor inmediato y crecimiento al producto.
- **MVF (Minimum Viable Feature)**: Son nodos translúcidos con alta incertidumbre. Son funciones "pioneras" para probar áreas nuevas; esencialmente, son versiones "mini" de un MVP ejecutadas dentro de un producto que ya existe.

“Mínimo” no está relacionado al tamaño, sino a la base necesaria para desarrollar algo funcional.

Cronología de lanzamiento: El ecosistema de los mínimos



Fases:

1. **MVP (aprendizaje)**: Múltiples iteraciones pequeñas y rápidas (prototipos).
2. **MMP / MMR 1 (Lanzamiento)**: El primer Release comercializable dirigido a early adopters. Contiene los MMFs esenciales.
3. **MMR 2... N (Evolución)**: Incrementos sucesivos que ofrecen características nuevas de valor continuado.

MLP (Mínimo Producto Adorable): no me alcanza con que la idea sea de un producto que lo van a utilizar; teniendo en cuenta que en el mercado hay otros competidores, aún cuando hubiera otro producto de características similares, quiero que mis usuarios elijan mi producto por sobre el resto. ¿Cómo lograrlo? reputación de la marca, el producto de software logra generar una conexión temprana con los usuarios, exploré de mejor manera la adopción inicial de ciertos usuarios, destaca en un mercado donde lo simplemente "funcional" ya es un estándar mínimo.

MVP (Producto Mínimo Viable)	MMP (Producto Mínimo Comercializable)	MLP (Producto Mínimo Adorable)
Enfoque: Entender necesidades Validar conceptos rápido y barato.	**Enfoque:** Amplio alcance / Atender a grandes bases de usuarios.	**Enfoque:** Diseñar para el deleite / Conexión emocional.
Foco Clave: Investigación, Código base, Pruebas de usuario.	**Foco Clave:** Estabilidad técnica, Lanzamiento al mercado.	**Foco Clave:** UI avanzada, Impacto emocional, Reducción de fricción.
Resultado: Informar el alcance del producto.	**Resultado:** Atracción de mercado masivo.	**Resultado:** Lealtad temprana y retención.

El ingeniero de calidad debe diagnosticar en qué fase se encuentra el producto para aplicar las métricas correctas.

Matriz diagnóstica: El espectro de los Productos Mínimos

	MVP (Viable)	MMP (Comercializable)	MLP (Adorable)
Objetivo Principal	Validar concepto de forma económica	Atender a base amplia con funcionalidad clave	Crear conexión emocional e interacción profunda
Enfoque de Diseño	Investigar y empatizar	Construir y refinar / Código y testing	Diseñar para el deleite / Impacto emocional
Beneficio Principal	Informar el alcance del producto	Asegurar la calidad y salida al mercado	Aumentar la lealtad y el amor del usuario
Ejemplo Típico	Entrevistas y videos	Lanzamiento Beta	App con animaciones UI divertidas y personalizadas

Requerimientos Ágiles en la práctica

A diferencia de las metodologías tradicionales donde los requisitos se documentan en especificaciones extensas (ej. ERS) y cerradas antes de programar, en la filosofía ágil el software se construye de a poco mediante incrementos pequeños y funcionales. No se entrega todo al final, sino que se conversa continuamente con el cliente, priorizando siempre aquello que le aporta mayor valor de negocio.

Historias de Usuario

- ➢ La importancia de los roles

El rol responde a la pregunta de para quién estamos construyendo el software y quién se beneficia de él. No se debe utilizar la palabra genérica "Usuario" si el dominio permite mayor precisión. Los roles deben ser inventados o adaptados específicamente al negocio que se está modelando (por ejemplo, en un sistema clínico: Médico, Secretaria, Paciente; en una app de gastos: Ejecutor de gastos, Responsable del gasto). Definir el rol con claridad permite entender exactamente qué valor de negocio se va a entregar y a quién, lo que ayuda a diseñar la funcionalidad correcta.

- ➢ La tarjeta de una historia de usuario

En la práctica cada historia de usuario se registra en una breve tarjeta con 4 partes.

1. El relato: La frase "Como... quiero... para..." que resume la necesidad en el lenguaje del negocio.
2. Criterios de aceptación: Reglas puntuales que la funcionalidad debe cumplir para considerarse terminada.
3. Pruebas de usuario: Casos (que fallan o pasan) que van a verificar que se cumplan esos criterios.
4. Estimación: Un número (story points) que resume la complejidad, el esfuerzo y la incertidumbre.

Jerarquía de Requerimientos y Spikes

Para organizar el trabajo y el Backlog, los requerimientos se agrupan en distintos niveles de abstracción:

- Tema: Es un conjunto de épicas o historias de usuario fuertemente relacionadas entre sí, que suelen agruparse para facilitar la planificación global.
- Épica: Es una necesidad muy grande y difícil de estimar. Una épica no cumple con el modelo INVEST (no es pequeña ni fácilmente estimable) y, por definición, jamás puede entrar completa en un solo Sprint. Con el tiempo, debe romperse en historias más chicas.
- Spike: Se utiliza cuando la incertidumbre (técnica o funcional) es demasiado alta para poder estimar el trabajo. Un Spike no es una entrega de software funcional, sino una investigación, análisis o Prueba de Concepto (PoC). Su único objetivo es quitar o reducir la incertidumbre. No se estima en Story Points ni tiene pruebas de aceptación clásicas; se le asigna un tiempo/esfuerzo límite (Timebox) para investigar y resolver la duda.

Criterios de Aceptación

Son las reglas puntuales y los limitantes que la funcionalidad debe cumplir obligatoriamente para considerarse terminada. Definen las "reglas de negocio" específicas de esa historia. Suelen redactarse de forma clara, muchas veces en voz pasiva (ej. "Se debe poder filtrar por fecha"). En estos criterios se define qué datos son obligatorios, cuáles son opcionales y cuáles son los valores por defecto (ej. "El sistema debe mostrar por defecto la fecha actual, permitiendo su modificación"). También incluyen restricciones de diseño o requerimientos no funcionales (ej. "La sesión del usuario no debe caducar").

Pruebas de Aceptación (Pasa/Falla)

Las pruebas (o casos de prueba) verifican empíricamente que los criterios de aceptación se cumplan. No validan si la funcionalidad "es linda", sino si el software respeta las reglas escritas. La convención para redactarlas es iniciar con la palabra Probar, seguida de la acción principal y la condición específica. Se dividen en dos resultados esperados:

- Pasa (Camino feliz): Probamos una acción válida y el sistema responde correctamente. Ej: "Probar registrar un gasto ingresando un monto positivo y una fecha actual -> Pasa".
- Falla (Camino negativo): Probamos una acción inválida y esperamos que el sistema nos frene o nos dé un error. Es vital probar que el software está preparado para bloquear acciones no permitidas. Ej: "Probar consultar un gasto filtrando por un periodo que incluya fechas futuras -> Falla".

Gestión y Procesos en el Desarrollo de Software

La creación de sistemas informáticos es una actividad intrincadamente ligada al esfuerzo humano. Para organizar este trabajo, la disciplina establece estructuras, ciclos y herramientas específicas.

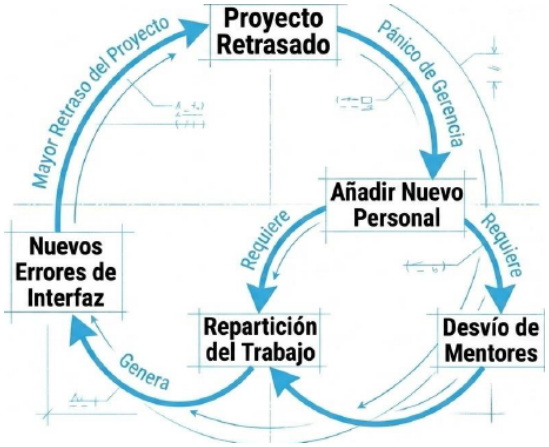
Producto: se desarrolla a través de un proceso estructurado que abarcará toda su vida útil, desde la idea hasta su operación.
Proyecto: es un esfuerzo puramente temporal destinado a crear o agregar funcionalidades a ese producto.

Un mismo producto de software invariablemente requerirá múltiples proyectos a lo largo de sus años de existencia.

Automatización: aplica herramientas para hacer que la ejecución del proceso sea más sencilla y efectiva. El objetivo es liberar al equipo de preocuparse por accidentes o tareas manuales menores.

Estimación y Organización de Equipos
El modelo mental de estimación humano suele fracasar porque confunde el esfuerzo invertido con el progreso real. Esto da origen al peligroso mito del hombre-mes, asumiendo que el tiempo y las personas son variables matemáticas intercambiables.

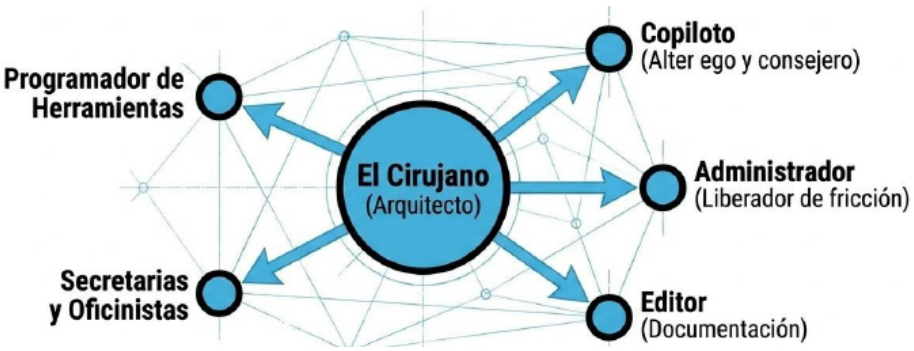
- **Ley de Brooks**: dictamina que añadir personal a un proyecto de software retrasado lo retrasará aún más. Esto sucede por el impacto brutal del tiempo de capacitación y la fricción operativa que recae sobre el equipo original.



El esfuerzo de comunicación crece de manera exponencial al sumar integrantes, devorando las ventajas de tener más trabajadores. Este problema combinatorio se calcula con la fórmula matemática.

Arquitectura e Integridad Conceptual
La **integridad conceptual** es la meta suprema del diseño de sistemas. Es preferible omitir funciones anómalas antes que construir un sistema plagado de ideas descoordinadas.

El **modelo del Cirujano** propone que la arquitectura fluya de una sola mente maestra, respaldada por ayudantes (copilotos, administradores, editores). Esta estructura logra la productividad de muchos reduciendo drásticamente la comunicación cruzada.



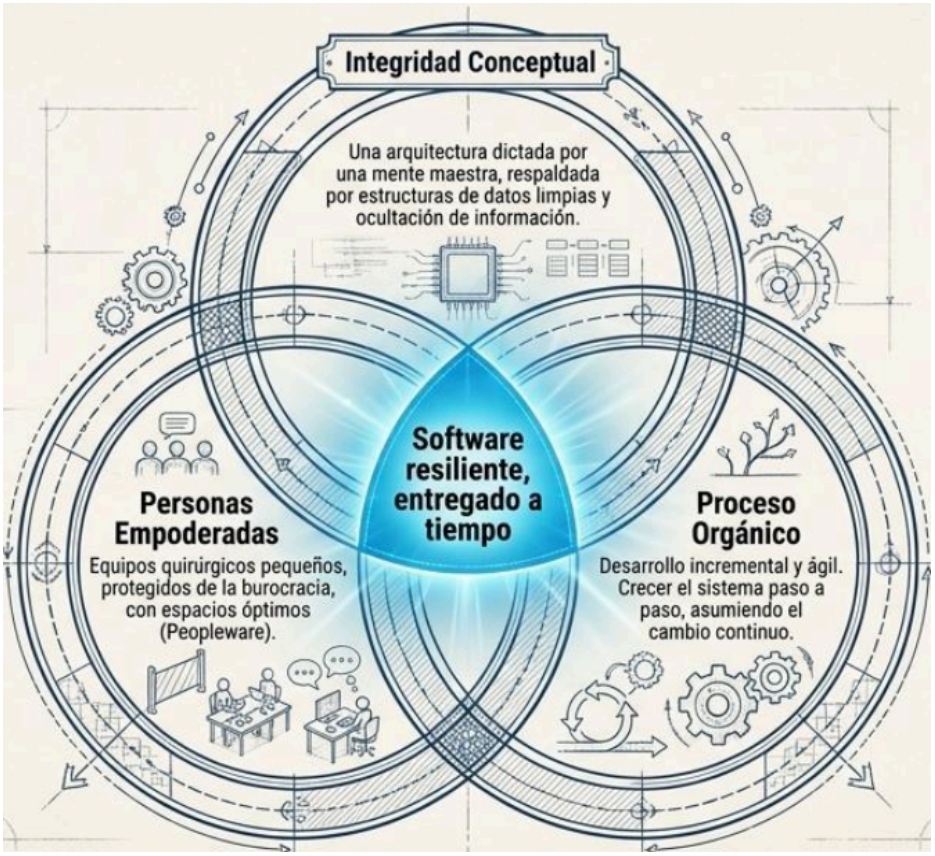
"*El sistema más peligroso es el segundo que construyes*"

- Síntomas:
  - ★ Pérdida del enfoque en el usuario principal.
  - ★ Sacrificio de velocidad por funciones rara vez utilizadas (featuritis).
  - ★ Manuales que engordan exponencialmente.
- Solución arquitectónica: asignar valores a priori en bytes o microsegundos a cada función. Mantener una vigilancia implacable sobre el presupuesto de recursos.

El enfoque de programación correcto exige rediseñar las tablas y estructuras de datos para que sean limpias y claras. La representación es la esencia de la programación.

La Hipótesis Documental: postula que un pequeño manual o cuaderno de trabajo actúa como el radar y pivote del proyecto. El simple acto de escribir este documento expone políticas difusas y mantiene la vigilancia del estado real de la arquitectura.

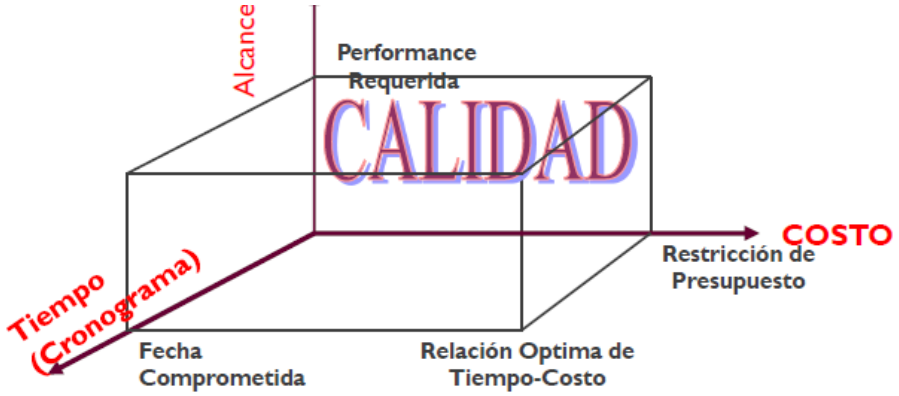
El ecosistema del desarrollo exitoso:

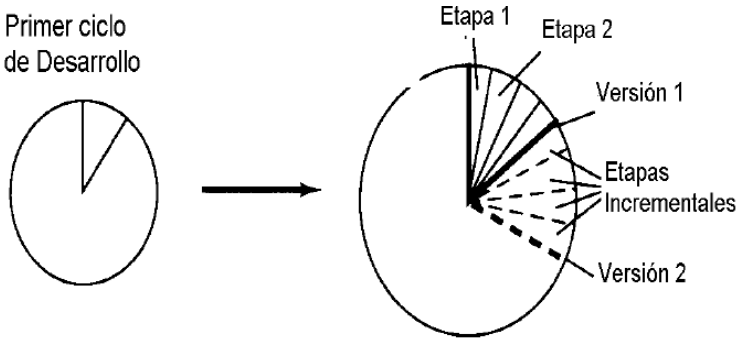


Gestión y Planificación de Proyectos

- **Administración de proyectos**: es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto. Incluye:
  - ★ Identificar los requerimientos.
  - ★ Establecer objetivos claros y alcanzables.
  - ★ Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders).
- **Líder de Proyecto**: es el encargado de balancear los objetivos, adaptar los enfoques a los diferentes stakeholders y lograr que el trabajo se haga.

- **La restricción triple**: es el balance constante entre los objetivos de alcance, el tiempo calendario y los costos del proyecto. La correcta gestión de estos tres factores es lo que impacta directamente en la calidad final del producto entregado.





- **Plan de proyecto**: es un documento vivo que define qué haremos, cuándo, cómo y quién se encargará. Debe ser capaz de absorber la mutabilidad del entorno.
  Implica:
  - ★ Definición del Alcance del Proyecto
  - ★ Definición de Proceso y Ciclo de Vida
  - ★ Estimación
  - ★ Gestión de Riesgos
  - ★ Asignación de Recursos
  - ★ Programación de Proyectos
  - ★ Definición de Controles
  - ★ Definición de Métricas
- **Alcance del producto**: son todas las características que pueden incluirse en un producto o servicio y se mide contra la Especificación de Requerimientos.
- **Alcance del proyecto**: es todo el trabajo y solo el trabajo que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas. Se evalúa midiendo contra el Plan de Proyecto.

Estimaciones de software:
- Tamaño
- Esfuerzo
- Calendario
- Costo
- Recursos críticos

Riesgo: es un evento latente o problema potencial que podría llegar a comprometer el éxito del trabajo.

Gestión de riesgos: consiste en identificarlos calculando su exposición (probabilidad de ocurrencia por impacto) para crear planes de contingencia o prevención.

Monitoreo y control: se encarga de comparar rigurosamente lo que se planificó frente a lo que está sucediendo en la realidad. Para realizar esto con precisión, la industria utiliza diferentes métricas.

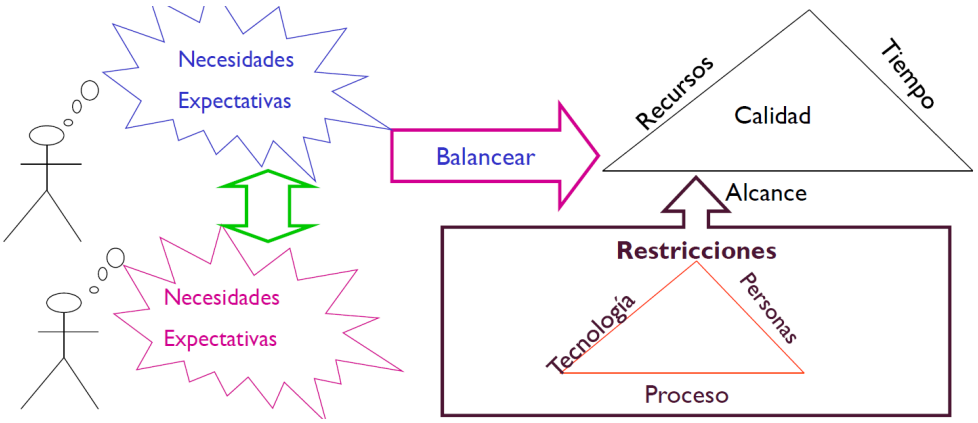
Métricas de producto: evalúan factores tangibles del entregable, como su tamaño final y la cantidad de defectos encontrados.

Métricas de proyecto: miden la eficiencia temporal, contrastando el esfuerzo gastado versus el planificado y el tiempo de calendario.

Métricas de proceso: consolidan los datos a lo largo de varios proyectos para evaluar el rendimiento histórico de toda la organización.

Métricas básicas para un proyecto de software:

- Tamaño del producto.
- Esfuerzo.
- Tiempo (calendario).
- Defectos.



Las métricas NUNCA deben utilizarse para medir el desempeño individual de los programadores. Si el equipo siente esto, comenzará a "trabajar para la métrica" manipulando los datos, en lugar de esforzarse por construir un buen software.

Causas comunes de fracaso:

- Fallas al definir el problema.
- Planificar basado en datos insuficientes.
- La planificación la hizo el grupo de planificaciones.
- No hay seguimiento del plan de proyecto.
- Plan de proyecto pobre en detalles.
- Planificación de recursos inadecuada.
- Las estimaciones se basaron en “supuestos” sin consultar datos históricos.
- Nadie estaba a cargo.

El fracaso de un proyecto rara vez es tecnológico; suele originarse en fallas en la comunicación, pobre definición inicial del problema o falta de seguimiento al plan.

Las estimaciones basadas en supuestos condenan al proyecto desde el inicio si no se consultan los datos y métricas históricas de la organización. Otro error crítico es que la planificación la realice un grupo externo que no participa ni se hace cargo de la ejecución.

Cultura ágil
Requiere un cambio en el comportamiento y mentalidad de la organización para poder implementarla con éxito. El motor de Scrum es un proceso puramente empírico que basa su aprendizaje en la experiencia directa frente a entornos cambiantes, complejos y con demasiadas variables.

Existen diferencias claras entre un **marco de trabajo** y una **metodología estricta**, siendo Scrum un framework sostenido por las bases del empirismo y el manifiesto ágil.

El manifiesto ágil establece un acuerdo de valores y principios orientados a priorizar lo que realmente aporta valor al momento de desarrollar software.

- **Valores**: cuáles son las cosas que debemos priorizar a la hora de desarrollar software.
- **Principios**: son como los valores pero con un poco más de detalle, un doble click.

Tienen que ver mucho con la cultura organizacional.

SCRUM 2026
Es un framework.

- **Framework**: marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.
- **Metodología**: interna prescribir una serie de pasos a seguir.

Tiene una estructura de valores y principios. Estos tienen como patrones una base del empirismo.

Los principios de Scrum: son seis bases operativas clave.

1. **Control de proceso empírico**: funciona con procesos empíricos porque aprende a partir de la experiencia. Mismos pilares. El concepto de transparencia es una cuestión cultural de la organización.
2. **Autoorganización**: sus equipos deben poder autogestionarse, no hay un líder de proyecto que organice tareas.
3. **Colaboración**: todas las personas que forman parte del equipo colaboran entre sí, así como PO, Scrum master, etc.
4. **Priorización basada en valor**
5. **Desarrollo iterativo**
6. **Time boxing**: unidades de tiempo acotadas para todos los eventos que plantea.

Es vital trabajar enfocados en los elementos que están en la cima de la pila de producto para ser efectivos. Especificar requerimientos de manera muy temprana genera un enorme desperdicio debido a la alta probabilidad de que existan cambios.

Motor empírico y el ciclo OODA:
Scrum se sostiene sobre la relación inseparable de tres pilares fundamentales que garantizan su funcionamiento.

- **Transparencia**: permite revelar la realidad de los procesos y del producto tal como son.
- **Inspección**: consiste en observar continuamente para detectar variaciones o situaciones emergentes. Para que esta inspección funcione y no pierda sentido, la condición innegociable es que exista una transparencia previa.

- **Adaptación**: es la acción de realizar un ajuste inmediato tras los hallazgos de la inspección. Si nosotros no hacemos adaptación, todo el esfuerzo previo de inspección se desperdicia por completo.

Este enfoque empírico se vincula directamente con la lente OODA (v2026), una estructura temporal que aporta un marco para la toma de decisiones.
Dentro de esta lente, observar requiere apertura y respeto para reunir evidencia honesta. La acción de orientar exige el coraje necesario para interpretar la realidad desafiando suposiciones.
Al momento de decidir, se deben plantear experimentos paralelos que sean seguros para fallar (safe-to-fail). Finalmente, actuar implica un compromiso férreo con la ejecución de lo pautado.
Tiene cuatro aristas:

1. **Observar**: observar la realidad para saber lo que ocurre.
2. **Orientar**: interpretar la realidad.
3. **Decidir**: qué hacer sobre esta interpretación de la realidad.
4. **Actuar**: compromiso de ejecución.

Valores de SCRUM:
El marco de trabajo depende de vivir diariamente sus valores, ya que la confianza es la moneda de cambio fundamental del equipo. Sin el coraje para decir "no" y el respeto para criticar las ideas en lugar de a las personas, el empirismo fracasa.

1. La **apertura y el respeto** sirven para integrar perspectivas diversas y obtener métricas honestas del proceso.
2. El **coraje** es la herramienta que permite navegar a través de la incertidumbre.
3. El **compromiso** garantiza la ejecución orientada siempre hacia el Sprint Goal dentro de las restricciones de tiempo establecidas.
4. El **foco** permite al equipo seleccionar únicamente los experimentos viables y priorizar el valor descartando aquello que no es relevante.

Célula profesional y su autogestión:
La unidad central de trabajo es el Scrum Team, una célula profesional autogestionada, pequeña y cognitivamente diversa. Esta polinización cruzada de habilidades es esencial y obligatoria para poder resolver problemas de alta complejidad.

- **Autogestión**: significa que el equipo decide internamente quién, qué, cómo, cuándo y dónde se ejecuta el trabajo. Bajo este concepto, se elimina y rechaza cualquier tipo de microgestión externa.
- **Alto profesionalismo**: implica tomar la responsabilidad total del ciclo de vida del producto. Esto abarca desde la concepción hasta el retiro del producto (de la cuna a la tumba), exigiendo siempre excelencia técnica y ética.

Actualmente, la IA se integra como un compañero de equipo más en los procesos de descubrimiento y entrega. Scrum se alinea al uso de estas herramientas tecnológicas asumiendo que no debemos quedarnos estrictamente vinculados a un plan manual.
Se exige un uso ético de la IA que ayude a validar hipótesis de negocio y reducir sesgos cognitivos. Bajo ningún concepto la IA debe utilizarse para reemplazar el juicio profesional del equipo de desarrollo.

Roles en la Guía Expandida

Rol	Naturaleza	Responsabilidad Principal
Product Owner	Humano	Maximizar el valor a largo plazo; gestionar e Product Backlog y la estrategia del producto.
Product Developer	Humano o Automatizado	Crear incrementos utilizables; adherirse a la Definición de Salida Terminada; autogestionar el plan del Sprint.
Scrum Master	Humano	Agente de cambio; guía la efectividad del equipo y la organización en la adopción de Scrum.
Stakeholder	**Humano/Entidad/IA**	Entidades interesadas o afectadas por el producto (clientes, usuarios, patrocinadores, legisladores).
Supporter	**Stakeholder específico**	Coalición poderosa que elimina impedimentos organizacionales y fomenta un clima para la autogestión.

- **Product Owner**: es el dueño del "qué" y el "por qué" del producto a construir. No es un simple escriba de tickets de requerimientos ni un comité, sino el único responsable de la narrativa de valor.
- **Scrum Master**: se enfoca en la efectividad del entorno actuando como un líder servicial. No es un secretario del equipo ni un administrador de Jira, su misión real es remover barreras organizacionales.
- **Product Developers**: son los dueños absolutos del "cómo" y de la calidad técnica del incremento. Su mentalidad se rige por el mantra de dejar de empezar tareas nuevas para enfocarse exclusivamente en terminar lo iniciado (stop starting, start finishing).
- **Supporters**: son aquellos gerentes que apoyan activamente la agilidad proveyendo recursos y un clima seguro. Si un manager directivo no empodera al equipo ni se encarga de remover impedimentos, no clasifica como Supporter.

Liderazgo y Cambio Organizacional

El liderazgo para la resiliencia es una conducta orientada a crear claridad y un entorno seguro para la creatividad. Se entiende como una actitud catalizadora y no como un cargo jerárquico tradicional.

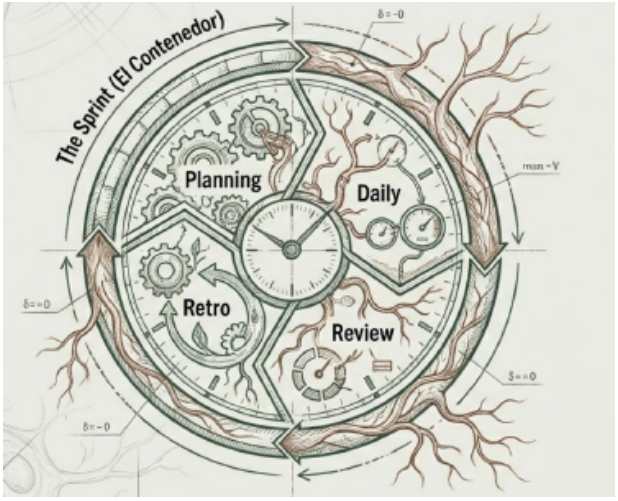
El cambio organizacional es un proceso netamente emergente donde solo se puede guiar la dirección del viaje. Es logísticamente imposible forzar un destino fijo en entornos de trabajo tan variables.

El Sprint y sus Bucles de Inspección

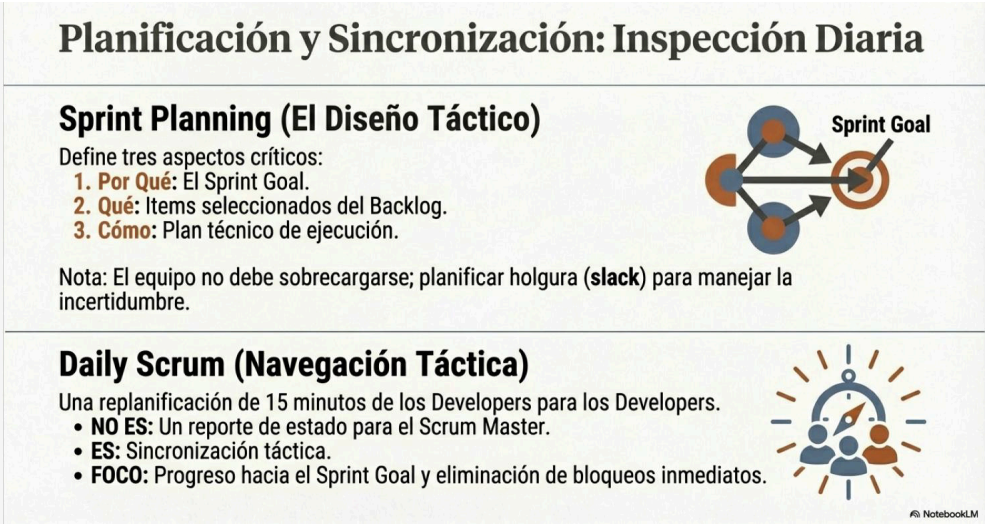
El Sprint funciona como un contenedor de consistencia seguro para fallar que aporta estabilidad dentro del caos del mercado. Su principal virtud es que prioriza la frecuencia de aprendizaje y validación por encima de la velocidad pura de entrega.
Establecer una cadencia fija crea hábitos de trabajo saludables en el equipo. Esta repetición constante reduce drásticamente la complejidad cognitiva de organizar el trabajo.
Trabajar con iteraciones cortas otorga un gran control de riesgo, ya que brindan oportunidades rápidas para pivotar la estrategia. A menor duración del ciclo de desarrollo, menor será el costo financiero de cometer un error.

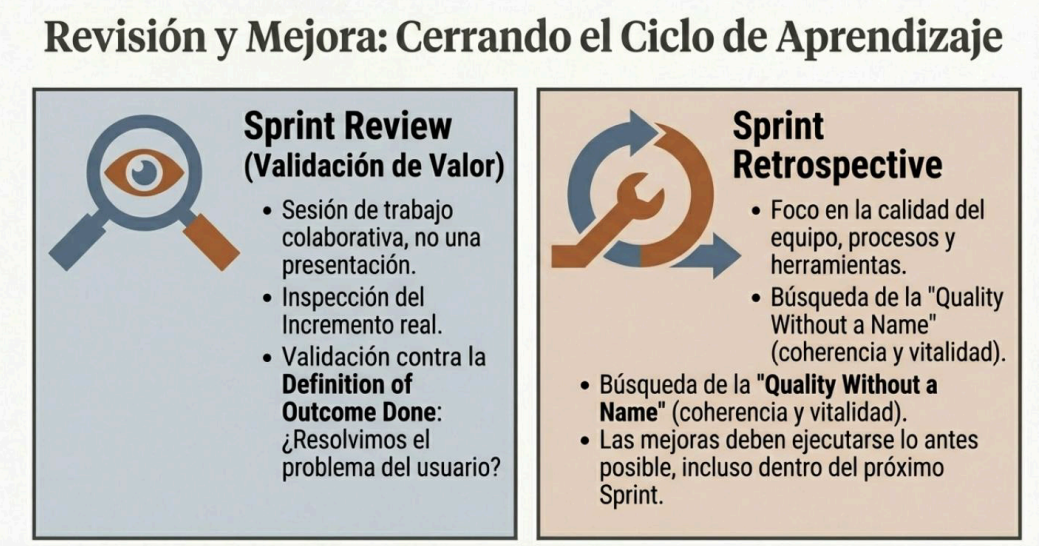
- **Regla de oro de la iteración**: inmutabilidad del objetivo. Mientras que el alcance (scope) puede llegar a negociarse si surge un nuevo aprendizaje, el Sprint Goal es completamente inamovible una vez iniciado el ciclo.

Eventos y Timeboxing Estándar



- **Sprint Planning**: define el diseño táctico integral del ciclo de trabajo. Determina el porqué a través del Sprint Goal, el qué mediante los ítems seleccionados y el cómo a través del plan técnico de los Developers (Forecast). El equipo planifica para terminar temprano para tener margen de maniobra ante la incertidumbre; asegurándose de no sobrecargarse jamás al máximo de su capacidad teórica.
- **Daily Scrum**: sesión de replanificación de 15 minutos exclusiva de los Developers para los Developers. Su enfoque primario es garantizar el flujo de trabajo hacia el Sprint Goal y exponer cualquier bloqueo inmediato. La Daily no es un reporte de estado para el Scrum Master, es puramente una sincronización táctica del equipo de construcción.
- **Sprint Review**: sesión de trabajo colaborativa con los Stakeholders orientada a validar el valor generado. Se inspecciona el incremento de producto real para determinar si efectivamente se resolvió el problema del usuario de negocio.
- **Sprint Retrospective**: busca desafiar las suposiciones operativas y mejorar la coherencia del sistema de trabajo. Todas las mejoras descubiertas en la calidad o en las herramientas deben aplicarse lo antes posible, incluso durante el próximo Sprint.





Pensamiento de Producto y Artefactos

- **Product Thinking**: entiende que los productos construidos son activos vivos en constante evolución. Esto difiere de la mentalidad de proyecto tradicional donde todo finaliza abruptamente en una fecha de entrega.
- **Satisfaction Gap (Discovery)**: representa la distancia entre la experiencia actual del usuario y su experiencia ideal deseada.
- **Crossing the Chasm (Delivery)**: estrategia enfocada en pasar de los usuarios tempranos a la mayoría pragmática del mercado.
- **Refinamiento Continuo**: es la actividad colaborativa esencial para descomponer ítems y reducir su nivel de incertidumbre. Esta tarea acepta que nuestras suposiciones pueden ser erróneas, integrando la investigación y validación previa antes de codificar.

El Discovery y el Delivery no ocurren en fases separadas. El descubrimiento de las necesidades del producto ocurre en simultáneo y dentro de la ejecución de cada Sprint.

Artefactos y Compromisos: La Estructura de la Transparencia



El **Product Backlog** es la lista viva, emergente y priorizada por valor de todo lo que el producto necesita. Su compromiso inquebrantable es el **Product Goal**, un corte vertical de la visión que define el "por qué" estratégico a mediano plazo.

El **Sprint Backlog** representa el plan táctico y detallado de los Developers para la iteración en curso. Su compromiso asociado es el **Sprint Goal**, el objetivo único y coherente que guía los esfuerzos del equipo durante esos días.

El **Increment** es la pieza de software o producto concreta y utilizable que se ha generado. Su compromiso formal es la **Definition of Done**, el estándar de calidad estricto que se exige para poder liberar ese trabajo al mercado.

El Triple Filtro: Output, Outcome y Ready (SGEP 2026)
La madurez ágil exige dejar de medir únicamente el código producido para comenzar a auditar el impacto de negocio logrado con ese esfuerzo.

Criterio Evaluado	Estado Ready (Preparación)	Output Done (Calidad Técnica)	Outcome Done (Valor de Negocio)
Propósito de uso	Reduce el riesgo y fomenta la claridad antes de empezar a construir.	Garantiza la calidad técnica y diligencia de construcción del Incremento.	Valida si se construyó el producto correcto midiendo la generación de valor.
Estatus en el SGEP	Cualidad emergente.	Compromiso formal innegociable.	Compromiso formal innegociable.
Artefacto Asociado	Product Backlog Item (PBI).	Increment.	Product.
Responsabilidad y Decisión	Scrum Team colaborativo (Los Devs son los únicos que estiman tamaño).	Developers (Accountability colectiva para no degradar el estándar).	Product Owner (Tiene la palabra final sobre la métrica de éxito).
Métrica Típica	Días necesarios o capacidad de comprensión del problema.	Cobertura de tests, seguridad y estándares de arquitectura limpios.	Telemetría, retorno de inversión (ROI) y cambio de comportamiento en el usuario.

- **Estado Ready**: cualidad emergente que indica que hay claridad absoluta y suficiente confianza para empezar el desarrollo. No representa un contrato formal, pero exige que los ítems estén desglosados, sin ambigüedades y dimensionados de forma factible para pocos días.
    - ★ Se debe ser muy cuidadoso de evitar el sobre-refinamiento, buscando apenas el nivel de detalle necesario para no generar desperdicio administrativo.
- **Definition of Output Done**: es un estándar de calidad técnico, formal y compartido que el equipo está obligado a cumplir. Bajo ninguna circunstancia se debe debilitar esta calidad por presión de tiempo; de ser necesario, se recorta el alcance, nunca la calidad.
    - ★ Entregar este Output perfecto no garantiza el éxito comercial, es el único habilitador real para poder recolectar feedback del cliente.

- **Definition of Outcome Done**: es la evidencia medible de que logramos generar un impacto de negocio o cambiar el comportamiento del usuario. Debe definirse antes de construir la funcionalidad para evitar sesgos analíticos al momento de revisar la telemetría.
    - ★ Existe un riesgo altísimo en los equipos inmaduros de confundir la entrega técnica (Output) con el éxito real (Outcome). La falta de liberaciones tempranas al mercado provoca una carencia total de evidencia observable que destruye el propósito empírico.

Gestión de Capacidad y Métricas Ágiles

El uso de métricas en ambientes ágiles debe entenderse como una salida de información para la mejora y no como una pesada actividad de reporte. La verdadera medida de progreso, según los principios ágiles, será siempre el software funcionando entregado tempranamente.

- La **capacidad del equipo** se calcula aplicando la fórmula de multiplicar las horas de trabajo disponibles () por los días de la iteración ().
  A esta capacidad teórica se le deben restar elementos ineludibles como el buffer del sprint, las ceremonias y las ausencias del personal. También es obligatorio descontar cualquier compromiso de soporte o mantenimiento de sistemas que sean externos al proyecto principal.
- La **Velocidad (Velocity)** mide el avance histórico sumando los story points de las US efectivamente completadas. Esta métrica se utiliza para corregir los errores de estimación a futuro y nunca contabiliza historias que quedaron parcialmente terminadas.
- Las **herramientas de seguimiento** (Taskboards, Sprint Burndown Charts, etc) ayudan a visibilizar este avance diario. Toda esta información visual da soporte a los múltiples niveles de planificación, desde el diseño de Portfolio a gran escala hasta la táctica de la Daily Scrum.

Dos principios ágiles que guían la elección de las métricas:

1. “Nuestra mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de software valioso”.
2. “El software trabajando es la principal medida de progreso”.